

# Strong Federated Authentication With Password-based Credential Against Identity Server Corruption

Changsong Jiang<sup>1,2</sup> , Chunxiang Xu<sup>1,3</sup>  , Guomin Yang<sup>2</sup> , Li Duan<sup>4</sup> , and Jing Wang<sup>5</sup> 

<sup>1</sup> School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu 611731, China

jiangchso@std.uestc.edu.cn, chxxu@uestc.edu.cn

<sup>2</sup> School of Computing and Information Systems, Singapore Management University, Singapore 188065, Singapore

gmyang@smu.edu.sg

<sup>3</sup> Yangtze Delta Region Institute (Huzhou), University of Electronic Science and Technology of China, Huzhou 313001, China

<sup>4</sup> Paderborn University, Paderborn, Germany

liduan@mail.upb.de

<sup>5</sup> School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan 430074, China

cswjing@hust.edu.cn

**Abstract.** We initiate the study of strong federated authentication with password-based credential against identity server corruption (SaPBC). We provide a refined formal security model, which captures all the necessary security properties in registration, authentication, and session key establishment between a user and an application server. The new model with fine-grained information leakage separates the leakage of password-related files and long-term secrets (including passwords and credentials). Moreover, we present two SaPBC protocols constructed from efficient cryptographic primitives for these corruption scenarios. In addition to rigorous security proofs, we also conduct comprehensive performance evaluation of the two protocols.

**Keywords:** Federated identities · Strong authentication · Password-based credential · Server corruption

## 1 Introduction

In a federated identity system, a user authenticates herself/himself to an identity server (IS) with a credential [4,35]. Once the user finishes the authentication successfully, she/he will receive a token from IS. The token can convince application servers of the user's legitimacy, such that the user can gain access to the associated services. Federated identities free users from maintaining redundant credentials for authentication to each individual service, thereby offering great

convenience. Such an authentication system has been drafted in standards such as OpenID [1], OAuth [2], and SAML [6]. In practice, passwords are the most common credentials due to their easy-operation: users merely need to memorize passwords to access services [25,26]. Nonetheless, passwords may provide a weak security guarantee. For example, Verizon’s 2024 Data Breach Investigations Report<sup>1</sup> shows that 77% of hacking-related data breaches arose from credential stolen (e.g., via phishing attacks against passwords) and 21% relied on brute-force attacks against weak passwords.

An alternative solution for authentication is to use a password-less approach such as strong authentication [31]. Through strong authentication, a user can prove her/his identity to IS by demonstrating knowledge of a particular secret via a cryptographic challenge-response mechanism. To guarantee security of strong authentication, it is crucial to protect the secret (i.e., credential). To this end, FIDO Alliance [15] recommended to store and operate the credential inside a tamper-resistant hardware module, such as YubiKey<sup>2</sup>. Although employment of tamper-resistant hardware is desirable in terms of security, it might not be ideal from the usability perspective [16,33]. For instance, a laboratory study [33] revealed that users generally viewed the usability of YubiKey poorly due to significant impediments to setting up accounts.

To address this issue, password-based credential (PBC) [36] was proposed, in which IS maintains a master secret and involves in an interactive protocol with each user to generate an individual credential. The credential is then protected (encrypted) with the user’s password. Without the knowledge of the master secret, the resulting password-protected credential is indistinguishable from a random group element. If an adversary  $\mathcal{A}$  only has the password-protected credential,  $\mathcal{A}$  cannot perform offline password guesses or forge new tokens. Therefore, general-purpose devices rather than tamper-resistant hardware can be used to store this credential. As shown in [36], the proposed PBC can be applied in federated identity systems, which frees end users from using tamper-resistant hardware and brings convenience.

Despite the appealing benefits, strong federated authentication with PBC in [36] still relies on a strong assumption that IS is always secure. Once an adversary  $\mathcal{A}$  corrupts IS and obtains the master secret,  $\mathcal{A}$  can generate any user’s credential and forge tokens to impersonate the user for service requests without knowing the real passwords. In reality, server corruption turns out to be a regular occurrence. Even large corporations, including Dropbox [20], Yahoo [21], and LinkedIn [22], have fallen victim to such attacks, resulting in great losses. Moreover, the mutual authentication of entities in [36], such as user-IS or user-(application server), are closely bound with Transport Layer Security (TLS) [32] and Public key Infrastructure (PKI) [11].

<sup>1</sup> <https://www.verizon.com/business/resources/Ta03/reports/2024-dbir-data-breach-investigations-report.pdf>.

<sup>2</sup> <https://docs.yubico.com/yesdk/users-manual/intro.html>.

**Table 1.** Functionality comparison between SaPBC and existing schemes.

|                                       | FIDO2 [7] | Strong federated<br>authentication with PBC [36] | PBC <sub>StE</sub> [16] | SaPBC |
|---------------------------------------|-----------|--------------------------------------------------|-------------------------|-------|
| Strong authentication of users        | ✓         | ✓                                                | ✓                       | ✓     |
| Resistance to credential leakage      | ✗         | ✓                                                | ✓                       | ✓     |
| Resistance to IS corruption           | ✓         | ✗                                                | ✓                       | ✓     |
| Authentication of application servers | ✗         | ✗                                                | ✗                       | ✓     |
| Session key negotiation               | ✗         | ✗                                                | ✗                       | ✓     |

### 1.1 Our Contributions

In this paper, we initiate the study of strong federated authentication with PBC against identity server corruption (SaPBC).

*Formal Definition with Augmented Functionalities.* We augment the existing notion of strong federated authentication with PBC [36] to support more functionalities. Note that there is a PBC scheme PBC<sub>StE</sub> [16] that can resist IS corruption. Our proposed SaPBC adopts the technique of PBC<sub>StE</sub> to achieve resistance against IS corruption in federated identities. Compared with PBC<sub>StE</sub>, SaPBC additionally achieves authentication of application servers and session key negotiation by using techniques such as Diffie-Hellman key agreement and authenticated encryption with associated data. A functionality comparison between SaPBC and existing schemes are shown in Table 1. More related work [10], [13], [23] is provided in Appendix A.

*Security Model with Fine-Grained Information Leakage.* We first provide a formal definition of SaPBC and its sub-protocols in the phases of registration, authentication, and AKE. Second, we propose a game-based security model to capture necessary security properties, especially the threat from various combinations of information leakage at different time point. Specifically, an adversary  $\mathcal{A}$  could control the Internet, tamper with transmitted messages, compromise (combinations of) entities' secrets, including password-related files, passwords, and credentials. Our new model covers the following security intuitions.

- S1 Unless in trivial cases, it is hard for  $\mathcal{A}$  to compute the session key established honestly, even if  $\mathcal{A}$  can see the password after the session finishes. The security property is formalized as key indistinguishability (see Definition 6).
- S2 Unless in trivial cases, such as obtaining all the necessary randomness, it is hard for an adversary  $\mathcal{A}$  to recover the password used by honest users, even if  $\mathcal{A}$  can see the session keys. The security property is formalized as password recovery resistance (see Definition 5).

*Efficient Protocols for Different Security Trade-offs.* We propose a basic SaPBC scheme and an enhanced one, aiming at achieving different security levels in either key exchange security or resistance against password recovery. For the new design, we strive to answer the two research questions below.

**Table 2.** Key indistinguishability achieved by our basic and enhanced SaPBC, where Level-0: IS is uncorrupt, Level-1: IS is corrupted after the AKE phase, Level-2: IS is corrupted and the ephemeral secrets of IS is leaked after the authentication phase.

|                | Level-0 | Level-1 | Level-2 |
|----------------|---------|---------|---------|
| Basic SaPBC    | ✓       | ✓       | ✗       |
| Enhanced SaPBC | ✓       | ✓       | ✓       |

F1 If the established credentials are put into use, how can session keys be efficiently established?

F2 From which phase on can SaPBC be decoupled from PKI and TLS? In another word, can the authentication and the (possible) key establishment be independent of TLS/PKI after a reliable initialization?

As illustrated in Table 2, the basic SaPBC scheme achieves the security of Level-0 and Level-1, while the enhanced one additionally achieves the security of level-2 with extra communication and computation overheads. The two schemes are constructed from efficient cryptographic primitives (e.g., pseudorandom functions, digital signature, public-key encryption, authenticated encryption with associated data, and Diffie-Hellman key agreement), and hence can be implemented with standard cryptographic libraries. The security proofs of the basic and enhanced SaPBC schemes and the performance evaluation demonstrate their security and efficiency.

## 1.2 Technical Overview

We highlight the critical security challenges and give an overview of the corresponding solutions we propose.

*Initialization against Password Guessing.* One of the critical operations in authenticated key exchange is the entity authentication, and this can rely on a signature key pair. In SaPBC, the key pair  $(vk_U, sk_U)$  is derived from  $U$ 's password  $pw$  and a key  $k$  (stored in  $U$ 's device) via a pseudorandom function and the key generation of digital signature.  $vk_U$  and the identity information is send to IS via a secure channel, while  $k$  are kept by the user. Moreover, an independent public-key encryption key pair  $(ek_U, dk_U)$  are generated. The encryption key  $ek_U$  are kept by the user, and  $dk_U$  is distributed to the IS. Note that the user kept only password-independent credentials  $k$  and  $dk_U$ , and IS does not have the  $k$  with high-entropy (See Fig. 4). Thus, after this initialization, compromising only one side (either the user or the corresponding IS) does not allow offline brute-force password guessing.

*Authentication and Authenticated Key Exchange with Forward Secrecy.* The backbone in SaPBC to establish session keys is Diffie-Hellmann key exchange (DH) [17]. To authenticate the IS without TLS/PKI, the  $ek_U$  is used by the user to encrypt a DH public key, which is also a challenge for IS. Only the IS with

the correct  $dk_U$  can recover the DH public key and compute the session key (See Fig. 5). The secrets for the session key depends on the DH ephemeral secrets which cannot be recovered by the adversary  $\mathcal{A}$ . Therefore, forward secrecy is also guaranteed even if  $\mathcal{A}$  sees  $pw$  or  $k$  afterwards, fulfilling S1, F1 and F2.

*Resistance against Password Recovery after Session Key Leakage.* Before service access,  $U$  and  $AS$  need to verify each other's identity with the aid of  $IS$ . To this end,  $U$  and  $AS$  will obtain shared messages once the corresponding authentication by  $IS$  passes. By using the shared messages, the two parties can accomplish mutual authentication and negotiate a shared session key for protection of subsequent communications. Thus, the session keys are always derived from ephemeral secrets that are unknown to the adversary and independent of  $pw$ , fulfilling S2 (See Fig 6).

## 2 Preliminaries

**Notations.** Let  $\lambda \in \mathbb{N}$  be the security parameter.  $\{0, 1\}^n$  denotes  $n$ -bit strings.  $|\mathcal{T}|$  denotes the number of elements in a finite set  $\mathcal{T}$ .  $\phi \xleftarrow{\$} \Phi$  represents that  $\phi$  is selected at random from  $\Phi$ .  $\Pr[e]$  denotes the probability that the event  $e$  occurs. Let  $\text{Funs}[\mathcal{D}, \mathcal{R}]$  denote the set of all functions from a domain  $\mathcal{D}$  to a range  $\mathcal{R}$ . We use PPT to represent probabilistic polynomial time. For a non-interactive algorithm  $G$ ,  $\text{out} \leftarrow G(\text{in})$  represents that  $\text{out}$  is the output of running  $G$  on input  $\text{in}$ . For an interactive algorithm/protocol  $\bar{G}$  executed between two parties  $\mathcal{P}_1$  and  $\mathcal{P}_2$ , we will use the notation  $(\text{out}_1, \text{out}_2) \leftarrow \bar{G}(\mathcal{P}_1(\text{in}_1), \mathcal{P}_2(\text{in}_2))$ , where  $\mathcal{P}_i$  ( $i \in \{1, 2\}$ ) has input  $\text{in}_i$  and outputs  $\text{out}_i$ .  $\text{out}_i$  is written as  $*$  when the output is not explicitly needed.

**Primitives.** We use standard cryptographic primitives such as pseudorandom function  $\text{PRF} : \mathcal{K} \times \{0, 1\}^n \rightarrow \mathcal{R}$  and digital signature  $\text{SIG} = \{\text{SIG.Gen}, \text{SIG.Sign}, \text{SIG.Vfy}\}$ . The key generation may be represented as  $(vk, sk) \leftarrow \text{SIG.Gen}(r)$ , which behaves deterministically when inputting an explicit random number  $r \in \{0, 1\}^\lambda$ .

We also use public-key encryption scheme  $\text{PKE} = \{\text{PKE.Gen}, \text{PKE.Enc}, \text{PKE.Dec}\}$ , authenticated encryption with associated data scheme  $\text{AEAD} = \{\text{AEAD.Gen}, \text{AEAD.Enc}, \text{AEAD.Dec}\}$ , message authenticated code scheme  $\text{MAC} = \{\text{MAC.Mac}, \text{MAC.Vry}\}$ , and Diffie-Hellman key exchange protocol. The complete syntax and security definition of the primitives can be found in Appendix B.

## 3 Strong Federated Authentication With PBC Against Identity Server Corruption (SaPBC)

### 3.1 SaPBC Definition

As depicted in Fig. 1, SaPBC involves three sorts of entities: users, an identity server ( $IS$ ), and application servers.

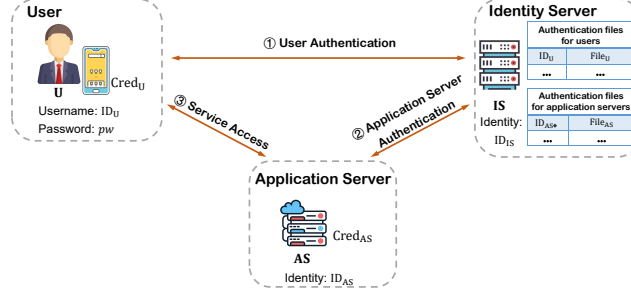


Fig. 1. System model.

- Users. Each user  $U$  equips with a general-purpose device, and has a username  $ID_U$  and a human-memorable password  $pw$ . After registering at IS,  $U$  stores her/his credential  $Cred_U$  in the device. When  $U$  attempts to access the services provided by AS,  $U$  first authenticates herself/himself to IS using  $pw$  and  $Cred_U$ . Then,  $U$  and AS execute service access, where the two parties can verify each other's legality if both of them are authenticated by IS, and negotiate a shared session key for protecting subsequent communications.
- Identity server (IS). IS is responsible for authenticating users and application servers. To identify a user  $U$ , IS checks if  $U$  possesses the password  $pw$  and the credential  $Cred_U$  corresponding to the authentication file  $File_U$  (aka password-related file). To identify AS, IS checks whether AS holds the credential  $Cred_{AS}$  that corresponds to the authentication file  $File_{AS}$ .
- Application servers. An application server AS is deployed by a service provider to provide users with services. After registering at IS, AS maintains a credential  $Cred_{AS}$ . Each time  $U$  attempts to access AS, AS will be authenticated by IS. Then, AS and  $U$  perform service access, where the two parties can check each other's validity if they are authenticated by IS, and build a shared session key.

**Definition 1 (SaPBC).** A SaPBC protocol  $\Sigma$  involves three phases: Registration, Authentication, and AKE, and consists of five subprotocols ( $\Sigma_{RegU}$ ,  $\Sigma_{RegAS}$ ,  $\Sigma_{AuthU}$ ,  $\Sigma_{AuthAS}$ ,  $\Sigma_{AKE}$ ) with username space  $\mathcal{D}_{uid}$ , application server's identity space  $\mathcal{D}_{asid}$ , and identity server's identity space  $\mathcal{D}_{isid}$ . Public parameters  $pp$  are generated and reliably distributed.

1) Registration.

- $((pw, Cred_U), File_U) \leftarrow \Sigma_{RegU}(U(ID_U), IS(*))$ . This subprotocol is run between user  $U$  with  $ID_U \in \mathcal{D}_{uid}$  and identity server IS.  $U$  outputs a password  $pw$  and a credential  $Cred_U$  and IS outputs an authentication file  $File_U$  for  $U$ .
- $(Cred_{AS}, File_{AS}) \leftarrow \Sigma_{RegAS}(AS(ID_{AS}), IS(*))$ . This subprotocol is executed between an application server AS with  $ID_{AS} \in \mathcal{D}_{asid}$  and IS. AS outputs a credential  $Cred_{AS}$  and IS outputs an authentication file  $File_{AS}$  for AS.

2) Authentication.

- $(m_U, \text{State}) \leftarrow \Sigma_{\text{AuthU}}\langle U(\text{ID}_U, \text{Cred}_U, pw, \text{ID}_{AS}), \text{IS}(\text{ID}_{IS}, \text{File}_U) \rangle$ . This subprotocol is executed between  $U$  and  $IS$ .  $IS$  inputs its identity  $\text{ID}_{IS}$  and the authentication file  $\text{File}_U$ , and outputs an ephemeral state  $\text{State}$ .  $U$  inputs her/his username  $\text{ID}_U$ , credential  $\text{Cred}_U$ , password  $pw$ , and the identity  $\text{ID}_{AS}$  of an application server that  $U$  attempts to access, and outputs a shared message  $m_U$  that will be used in the AKE phase.
  - $(m_{AS}, *) \leftarrow \Sigma_{\text{AuthAS}}\langle AS(\text{ID}_{AS}, \text{Cred}_{AS}), \text{IS}(\text{ID}_{IS}, \text{File}_{AS}, \text{State}, \text{ID}_U) \rangle$ . This subprotocol is run between  $AS$  and  $IS$ .  $IS$  takes as input its identity  $\text{ID}_{IS}$ , the authentication file  $\text{File}_{AS}$ , the ephemeral state  $\text{State}$ , and the username  $\text{ID}_U$ .  $AS$  inputs its identity  $\text{ID}_{AS}$  and credential  $\text{Cred}_{AS}$ , and outputs a shared message  $m_{AS}$  that will be used in the AKE phase.
- 3) AKE.
- $(SK_U, SK_{AS}) \leftarrow \Sigma_{\text{AKE}}\langle U(\text{ID}_U, m_U), AS(\text{ID}_{AS}, m_{AS}) \rangle$ . This subprotocol is performed between  $U$  and  $AS$ .  $U$  inputs  $\text{ID}_U$  and  $m_U$ , and  $AS$  inputs  $\text{ID}_{AS}$  and  $m_{AS}$ . The two parties output a shared session key  $SK_U = SK_{AS}$ .

**Definition 2 (Correctness).** *SaPBC is correct if for all  $\lambda \in \mathbb{N}$ ,  $\text{ID}_U \in \mathcal{D}_{\text{uid}}$ ,  $\text{ID}_{AS} \in \mathcal{D}_{\text{asid}}$ ,  $\text{ID}_{IS} \in \mathcal{D}_{\text{isid}}$ ,  $((pw, \text{Cred}_U), \text{File}_U) \leftarrow \Sigma_{\text{RegU}}\langle U(\text{ID}_U), \text{IS}(*) \rangle$ , and  $(\text{Cred}_{AS}, \text{File}_{AS}) \leftarrow \Sigma_{\text{RegAS}}\langle AS(\text{ID}_{AS}), \text{IS}(*) \rangle$ , the following holds:*

$$\Pr[SK_U = SK_{AS}] \geq 1 - \epsilon(\lambda), \quad (1)$$

where  $(SK_U, SK_{AS}) \leftarrow \Sigma_{\text{AKE}}\langle U(\text{ID}_U, m_U), AS(\text{ID}_{AS}, m_{AS}) \rangle$ ,  $(m_U, \text{State}) \leftarrow \Sigma_{\text{AuthU}}\langle U(\text{ID}_U, \text{Cred}_U, pw, \text{ID}_{AS}), \text{IS}(\text{ID}_{IS}, \text{File}_U) \rangle$ , and  $(m_{AS}, *) \leftarrow \Sigma_{\text{AuthAS}}\langle AS(\text{ID}_{AS}, \text{Cred}_{AS}), \text{IS}(\text{ID}_{IS}, \text{File}_{AS}, \text{State}, \text{ID}_U) \rangle$ .

### 3.2 Security Model

**Execution Environment.** The numbers  $d, \ell \in \mathbb{N}$  are positive integers. A set of  $d$  honest parties  $\{P_1, \dots, P_d\}$  is fixed in the execution environment. A SaPBC protocol involves participants from three different sub-sets (roles): the user set  $\mathcal{U}$ , the identity server set  $\mathcal{IS}$ , and the application server set  $\mathcal{AS}$ . We use an index  $i \in [d]$  to identify a party in the security experiment. Each participant  $P_i \in \{\mathcal{U}, \mathcal{IS}, \mathcal{AS}\}$  can have multiple instances to execute the protocol multiple times. For simplicity, we assume the role of each party is fixed.

We assume a strong man-in-the-middle adversary who fully controls the Internet and can tamper with every message on the network.

**Queries.** The queries are defined in Fig. 2 and the rationale is described briefly as follows.

- $\text{Send}(\pi_i^s, m)$ : This query models passive and active attacks.  $\mathcal{A}$  can send any message  $m$  to  $\pi_i^s$  and obtains the corresponding response.
- $\text{RevealKey}(\pi_i^s)$ : This query models leakage of session key(s).  $\mathcal{A}$  can obtain all session keys except the one of the target session.
- $\text{LeakFile}(P_i)$ : This query models leakage of authentication files on the  $IS$  side.
- $\text{LeakPW}(P_i)$ : This query leaks the password of a user  $P_i$ .

**Table 3.** Internal states of a session oracle  $\pi_i^s$ .

| Variable | Description                                                                                                   |
|----------|---------------------------------------------------------------------------------------------------------------|
| role     | records the role of the oracle $\pi_i^s$ in the session, $\text{role} \in \{\text{U}, \text{AS}, \text{IS}\}$ |
| PID      | records the identities $\{j\} \subset \{1, \dots, \ell\}$ of intended communication partner(s) $\{P_j\}$      |
| $\Phi$   | denotes $\Phi \in \{\text{fresh}, \text{exposed}\}$                                                           |
| st       | denotes $\text{st} \in \{\text{accept}, \text{reject}\}$                                                      |
| K        | records the session key $K \in \mathcal{SK}$ . $\mathcal{SK}$ : session key space, $\{0, 1\}^\lambda$         |
| $m_s$    | records the shared message of $\pi_i^s$                                                                       |

- $\text{LeakDK}(P_i)$  This query leaks the device key (i.e., the credential stored in the device) of a user  $P_i$ .
- $\text{Corrupt}(P_i)$ : This query returns the long-term secret of  $P_i$ , which may include password, device key, file or long-term secret key, depending on  $P_i$ 's role.
- $\text{RevealShMsg}(\pi_i^s)$ : This query models leakage of the shared message.
- $\text{Test}(\pi_i^s)$ : This query does not model any attack but evaluates the consequence of attacks: the (in)distinguishability of keys.

**Partnership and Trivial Predicates** On the one hand, we need to model an strong adversary which can tamper with messages and obtain all information “unrelated” to the target session. On the other hand, we would like to exclude the trivial attacks which no protocol can withstand, especially the information leakage from the partnered session. Therefore, we employ Li and Schäge’s key-based partnership definitions [30] and define the trivial predicates in Fig. 3 to exclude the trivial attacks against protocols in SaPBC. Typical trivial cases in AKE include direct revealing the established session key, compromising the involving parties in advance of authentication and key exchange, and a trivial attack against passwords is compromising both the credential of the user and the authentication files on the IS side (or reference value held by other parties).

**Definition 3 (Original key, Definition 2.1 in [30]).** *The original key of a pair of communicating oracles is the session key that is computed by each of the oracles in a protocol execution in the presence of an entirely passive adversary (which only relays the messages sent by the oracles).*

**Definition 4 (Original Key Partnering, Definition 4.1 in [30]).** *We say that two oracles  $\pi_i^s$  and  $\pi_j^t$  are said to be partnered if both of them have computed their original key.*

Let  $\mathcal{O} = \{\text{Send}, \text{RevealKey}, \text{LeakPW}, \text{LeakDK}, \text{LeakFile}, \text{Corrupt}, \text{Test}\}$ . Let  $\mathcal{O}_{\text{pw}} = \{\text{Send}, \text{RevealKey}, \text{LeakPW}, \text{LeakDK}, \text{LeakFile}, \text{Corrupt}\}$ . Let  $\Sigma$  be a protocol. In Fig. 2, We define two games:  $\mathbf{G}_{\mathcal{A}, \Sigma}^{\text{pw-recov}}(1^\lambda)$  for password guessing, and  $\mathbf{G}_{\mathcal{A}, \Sigma}^{\text{key-ind}}(1^\lambda)$  for indistinguishability of session keys and entity authentication.



|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\overline{G_{\mathcal{A}, \Sigma}^{\text{pw-recov}}(1^\lambda)}$ <pre> 1: <math>\mathcal{L}_{\text{qu}} \leftarrow \emptyset;</math> 2: <math>pw', P_i \leftarrow \mathcal{A}^{\mathcal{O}_{pw}(\cdot)}();</math> 3: <b>if</b> <math>\exists s : (\text{TrivialU-I dp}(\pi_i^s, \mathcal{L}_{\text{qu}})</math> 4:   <math>= (*, *, \text{true})) \vee</math> 5:   <math>(\text{TrivialPw}(\pi_i^s, \mathcal{L}_{\text{qu}})</math> 6:     <math>= (*, *, \text{true})) :</math> 7:   <b>return</b> 0; 8: <b>else</b> : <b>return</b> <math>(P_i.pw = pw');</math> </pre> $\overline{\text{Send}(\pi_i^s, m)}$ <pre> 1: <math>\mathcal{L}_{\text{qu}} \leftarrow \mathcal{L}_{\text{qu}} \cup \{\text{Send}(\pi_i^s, m)\}</math> 2: <b>if</b> <math>\pi_i^s</math> defined : 3:   <b>return</b> <math>\pi_i^s(m);</math> </pre> $\overline{\text{RevealKey}(\pi_i^s)}$ <pre> 1: <math>\mathcal{L}_{\text{qu}} \leftarrow \mathcal{L}_{\text{qu}} \cup \{\text{RevealKey}(\pi_i^s)\}</math> 2: <b>if</b> <math>\pi_i^s.\text{st} \neq \text{accept} :</math> <b>return</b> <math>\perp;</math> 3: <math>\pi_i^s.\Phi \leftarrow \text{exposed};</math> 4: <b>return</b> <math>\pi_i^s.K;</math> </pre> $\overline{\text{LeakFile}(P_i)}$ <pre> 1: <math>\mathcal{L}_{\text{qu}} \leftarrow \mathcal{L}_{\text{qu}} \cup \{\text{LeakFile}(P_i)\}</math> 2: <b>if</b> <math>P_i \notin \mathcal{IS} :</math> <b>return</b> <math>\perp;</math> 3: <math>E = \{\text{File}_U\}_{U \in \mathcal{U}};</math> 4: <b>return</b> <math>E;</math> </pre> $\overline{\text{Corrupt}(P_i)}$ <pre> 1: <math>\mathcal{L}_{\text{qu}} \leftarrow \mathcal{L}_{\text{qu}} \cup \{\text{Corrupt}(P_i)\}</math> 2: <math>\forall \pi_i^s</math> <b>and</b> <math>\pi_i^s.\text{st} \neq \text{accept} :</math> 3:   <math>\pi_i^s.\Phi \leftarrow \text{exposed};</math> 4: <b>if</b> <math>P_i \in \mathcal{U} :</math> <b>return</b> <math>(P_i.\text{Cred}, P_i.pw);</math> 5: <b>if</b> <math>P_i \in \mathcal{IS} :</math> <b>return</b> <math>(P_i.sk, E);</math> 6: <b>if</b> <math>P_i \in \mathcal{AS} :</math> <b>return</b> <math>P_i.sk;</math> </pre> | $\overline{G_{\mathcal{A}, \Sigma}^{\text{key-ind}}(1^\lambda)}$ <pre> 1: <math>\mathcal{L}_{\text{qu}} \leftarrow \emptyset;</math> 2: <math>b' \leftarrow \mathcal{A}^{\mathcal{O}(\cdot)}();</math> 3: <b>if</b> <math>\text{TrivAKE}(\pi_i^s, \mathcal{L}_{\text{qu}}) = \text{true} :</math> 4:   <b>return</b> 0; 5: <b>else</b> : <b>return</b> <math>(b = b') \text{ or}</math> 6:   <math>(\pi_i^s \text{ accepts maliciously});</math> </pre> $\overline{\text{LeakDK}(P_i)}$ <pre> 1: <math>\mathcal{L}_{\text{qu}} \leftarrow \mathcal{L}_{\text{qu}} \cup \{\text{LeakDK}(P_i)\}</math> 2: <b>if</b> <math>P_i \notin \mathcal{U} :</math> 3:   <b>return</b> <math>\perp;</math> 4: <b>return</b> <math>P_i.\text{Cred};</math> </pre> $\overline{\text{LeakPW}(P_i)}$ <pre> 1: <math>\mathcal{L}_{\text{qu}} \leftarrow \mathcal{L}_{\text{qu}} \cup \{\text{LeakPW}(P_i)\}</math> 2: <b>if</b> <math>P_i \notin \mathcal{U} :</math> 3:   <b>return</b> <math>\perp;</math> 4: <b>return</b> <math>P_i.pw;</math> </pre> $\overline{\text{Test}(\pi_i^s)} \quad // \text{ Allowed once only}$ <pre> 1: <math>\mathcal{L}_{\text{qu}} \leftarrow \mathcal{L}_{\text{qu}} \cup \{\text{Test}(\pi_i^s)\}</math> 2: <b>if</b> <math>\pi_i^s.\text{st} \neq \text{accept} \vee</math> 3:   <math>\pi_i^s.K = \perp :</math> 4:   <b>return</b> <math>\perp;</math> 5: <math>b \xleftarrow{\\$} \{0, 1\};</math> 6: <math>K_0 \leftarrow \pi_i^s.K;</math> 7: <math>K_1 \xleftarrow{\\$} \mathcal{SK};</math> 8: <b>return</b> <math>K_b;</math> </pre> $\overline{\text{TrivAKE}(\pi_i^s, \mathcal{L}_{\text{qu}})}$ <pre> 1: <b>if</b> <math>\text{TrivialCorr}(\pi_i^s, \mathcal{L}_{\text{qu}}) = \text{true} :</math> 2:   <b>return</b> <b>true</b>; 3: <b>return</b> <b>false</b>; </pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Fig. 2.** Security games and oracle definitions as pseudo-code. As a convention, 1 = true, 0 = false.

| TrivialU-Ildp( $\pi_i^s, \mathcal{L}_{qu}$ )                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | TrivialCorr( $\pi_i^s, \mathcal{L}_{qu}$ )                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> 1 :    // File and Cred allows pw guessing 2 :    if <math>P_i \in \mathcal{U} \wedge \exists \pi_j^t : P_j \in \mathcal{IS} \wedge</math> 3 :      Partner(<math>\pi_i^s, \pi_j^t</math>) = true : 4 :      if <math>\exists \tau, \tau' :</math> 5 :        <math>\mathcal{L}_{qu}[\tau] \in \{\text{Corrupt}(\pi_i^s),</math> 6 :          LeakDK(<math>\pi_i^s\}) \wedge \mathcal{L}_{qu}[\tau'] \in</math> 7 :          <math>\{\text{Corrupt}(\pi_j^t), \text{LeakFile}(P_j)\} :</math> 8 :          return <math>(\tau, \tau', \text{true})</math>; 9 :      if <math>P_i \in \mathcal{IS} \wedge \exists \pi_j^t : P_j \in \mathcal{U} \wedge</math> 10 :        Partner(<math>\pi_i^s, \pi_j^t</math>) = true : 11 :        if <math>\exists \tau, \tau' :</math> 12 :          <math>\mathcal{L}_{qu}[\tau] \in \{\text{Corrupt}(\pi_j^t),</math> 13 :            LeakDK(<math>\pi_j^t\}) \wedge \mathcal{L}_{qu}[\tau'] \in</math> 14 :            <math>\{\text{Corrupt}(\pi_i^s), \text{LeakFile}(P_i)\} :</math> 15 :            return <math>(\tau, \tau', \text{true})</math>; 16 :      return <math>(\perp, \perp, \text{false})</math>; </pre> | <pre> 1 :    if Test(<math>\pi_i^s</math>) <math>\notin \mathcal{L}_{qu} :</math> return true; 2 :    <math>\tau := \text{Index of Test}(\pi_i^s) \text{ in } \mathcal{L}_{qu};</math> 3 :    // Cred and pw leaked before Test() 4 :    <math>(\tau_1, \tau_2, b^*) \leftarrow \text{TrivialPw}(\pi_i^s, \mathcal{L}_{qu});</math> 5 :    if <math>b^* = \text{true}</math> 6 :      <math>\wedge (\tau_1 &lt; \tau) \wedge (\tau_2 &lt; \tau) :</math> 7 :      return true; 8 :    <math>(\tau_1, \tau_2, b^*) \leftarrow \text{TrivialU-Ildp}(\pi_i^s, \mathcal{L}_{qu});</math> 9 :    if <math>b^* = \text{true}</math> 10 :      <math>\wedge (\tau_1 &lt; \tau) \wedge (\tau_2 &lt; \tau) :</math> 11 :      return true; 12 :    // IS corruption before Test() 13 :    if <math>P_i \in \mathcal{IS}, \exists \tau' : \tau' &lt; \tau</math> 14 :      <math>\wedge \mathcal{L}_{qu}[\tau'] = \text{Corrupt}(\pi_i^s)</math> 15 :      return true; 16 :    // Target session key leakage 17 :    if <math>\exists \tau' : \tau' &lt; \tau</math> 18 :      <math>\wedge \mathcal{L}_{qu}[\tau'] = \text{RevealKey}(\pi_i^s) :</math> 19 :      return true; 20 :    if <math>\exists \pi_j^t, \tau' &lt; \tau :</math> 21 :      Partner(<math>\pi_i^s, \pi_j^t</math>) = true 22 :      <math>\wedge \mathcal{L}_{qu}[\tau'] = \text{RevealKey}(\pi_j^t) :</math> 23 :      return true; 24 :    // Shared message leakage 25 :    if <math>\exists \tau' : \tau' &lt; \tau</math> 26 :      <math>\wedge \mathcal{L}_{qu}[\tau'] = \text{RevealShMsg}(\pi_i^s) :</math> 27 :      return true; 28 :    if <math>\exists \pi_j^t, \tau' &lt; \tau :</math> 29 :      Partner(<math>\pi_i^s, \pi_j^t</math>) = true 30 :      <math>\wedge \mathcal{L}_{qu}[\tau'] = \text{RevealShMsg}(\pi_j^t) :</math> 31 :      return true; 32 :    return false; </pre> |
| TrivialPw( $\pi_i^s, \mathcal{L}_{qu}$ )                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <pre> 1 :    // Cred and pw allows 2 :    // user impersonation 3 :    if <math>P_i \in \mathcal{U}, \exists \tau, \tau' :</math> 4 :      <math>\mathcal{L}_{qu}[\tau] = \text{LeakPW}(\pi_i^s)</math> 5 :      <math>\wedge \mathcal{L}_{qu}[\tau'] = \text{LeakDK}(\pi_i^s) :</math> 6 :      return <math>(\tau, \tau', \text{true})</math>; 7 :    if <math>P_i \in \mathcal{U}, \exists \tau :</math> 8 :      <math>\mathcal{L}_{qu}[\tau] = \text{Corrupt}(\pi_i^s) :</math> 9 :      return <math>(\tau, \tau, \text{true})</math>; 10 :    return <math>(\perp, \perp, \text{false})</math>; </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

Fig. 3. Trivial predicates.

**Definition 5 (Password Recovery Resistance).** We say that a protocol  $\Sigma$  in SaPBC has  $(t, \epsilon)$ -resistance against password recovery, if there is a function  $\epsilon(\lambda)$ , for any adversary  $\mathcal{A}$  with running time upper bounded by  $t$ , it holds for the advantage of  $\mathcal{A}$  in  $\mathbf{G}_{\mathcal{A}, \Sigma}^{\text{pw-recov}}(1^\lambda)$  that:

$$\text{Adv}_{\mathcal{A}, \Sigma}^{\text{pw}} := \left| \Pr[\mathbf{G}_{\mathcal{A}, \Sigma}^{\text{pw-recov}}(1^\lambda) \Rightarrow 1] \right| \leq \epsilon(\lambda). \quad (2)$$

**Definition 6 (Key Indistinguishability).** A protocol  $\Sigma$  in SaPBC has  $(t, \epsilon)$ -key indistinguishability, if there is a function  $\epsilon(\lambda)$ , for any adversary  $\mathcal{A}$  with running time upper bounded by  $t$ , it holds for  $\mathcal{A}$ 's advantage in  $\mathbf{G}_{\mathcal{A}, \Sigma}^{\text{key-ind}}(1^\lambda)$  that:

$$\text{Adv}_{\mathcal{A}, \Sigma}^{\text{key}} := \left| \Pr[\mathbf{G}_{\mathcal{A}, \Sigma}^{\text{key-ind}}(1^\lambda) \Rightarrow 1] - \frac{1}{2} \right| \leq \epsilon(\lambda). \quad (3)$$

**Definition 7 (Authentication).** If there exists an oracle  $\pi_i^s$  with probability at least  $\epsilon(\lambda)$  such that there is no unique oracle  $\pi_j^t$  s.t.  $\pi_i^s$  and  $\pi_j^t$  are partnered in  $\mathbf{G}_{\mathcal{A}, \Sigma}^{\text{key-ind}}(1^\lambda)$ , then we say  $\pi_i^s$  accepts maliciously. A protocol  $\Sigma$  in SaPBC has  $(t, \epsilon)$ -authentication, if for any adversary  $\mathcal{A}$  with running time upper bounded by  $t$ , it holds that  $\mathcal{A}$  cannot make  $\pi_i^s$  accept maliciously.

## 4 Basic and Enhanced SaPBC: Constructions

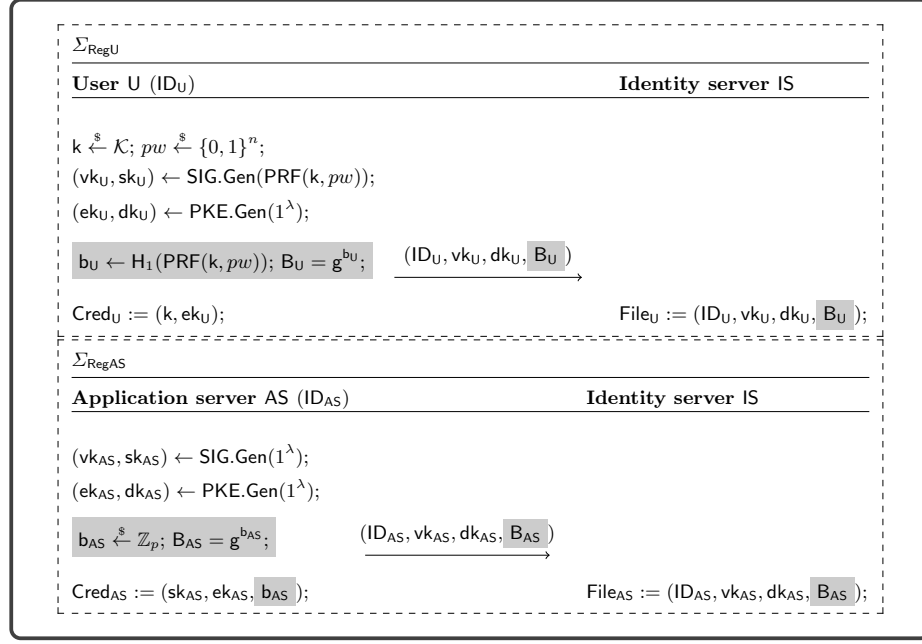
In this section, we provide detailed constructions of the basic SaPBC protocol  $\Sigma^B$  and the enhanced one  $\Sigma^E$  in Figs. 4, 5, and 6.

In the SaPBC protocol  $\Sigma$  ( $\Sigma = \Sigma^B$  or  $\Sigma^E$ ), public parameters  $pp = \{\mathbb{G}, p, g, \text{SIG}, \text{PKE}, \text{PRF}, \text{AEAD}, \text{MAC}, H_1, H_2, H_3, H_4\}$  are generated with the security parameter  $\lambda$  and securely distributed.  $\mathbb{G}$  is a multiplicative group of order  $p$  with generator  $g$ , SIG is a digital signature scheme, PKE is a public-key encryption scheme,  $\text{PRF} : \mathcal{K} \times \{0, 1\}^n \rightarrow \mathcal{R}$  is a pseudorandom function, AEAD is an authenticated encryption with associated data scheme, MAC is a message authenticated code scheme, and  $H_1 : \mathcal{R} \rightarrow \mathbb{Z}_p$ ,  $H_2 : \mathbb{G} \times \{0, 1\}^* \rightarrow \mathbb{Z}_p$ ,  $H_3 : \mathbb{G} \times \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ , and  $H_4 : \mathbb{G} \times \{0, 1\}^* \rightarrow \mathcal{SK}$  are hash functions.

1) *Registration.* The registration phase consists of two subprotocols  $\Sigma_{\text{RegU}}$  and  $\Sigma_{\text{RegAS}}$ , which are depicted in Fig. 4. In  $\Sigma_{\text{RegU}}$ , a user  $U$  with username  $\text{ID}_U$  registers with the identity server IS. In  $\Sigma_{\text{RegAS}}$ , an application server AS with identity  $\text{ID}_{AS}$  registers with IS.

2) *Authentication.* The authentication phase consists of two subprotocols  $\Sigma_{\text{AuthU}}$  and  $\Sigma_{\text{AuthAS}}$ , as shown in Fig. 5. In  $\Sigma_{\text{AuthU}}$ ,  $U$  authenticates to IS using password  $pw$  and credential  $\text{Cred}_U$ . In  $\Sigma_{\text{AuthAS}}$ , IS authenticates AS by verifying whether AS holds  $\text{Cred}_{AS}$ . In the two subprotocols, IS assists the valid  $U$  and AS in generating the shared message.

3) *AKE.* As illustrated in Fig. 6, the AKE phase consists of one subprotocol  $\Sigma_{\text{AKE}}$ , in which  $U$  and AS verify legitimacy of each other's identities using the shared message and negotiate a shared session key.



**Fig. 4.** Registration phase of the SaPBC protocol  $\Sigma$  ( $\Sigma = \Sigma^B$  or  $\Sigma^E$ ). The code in gray boxes is only executed in  $\Sigma^E$ .

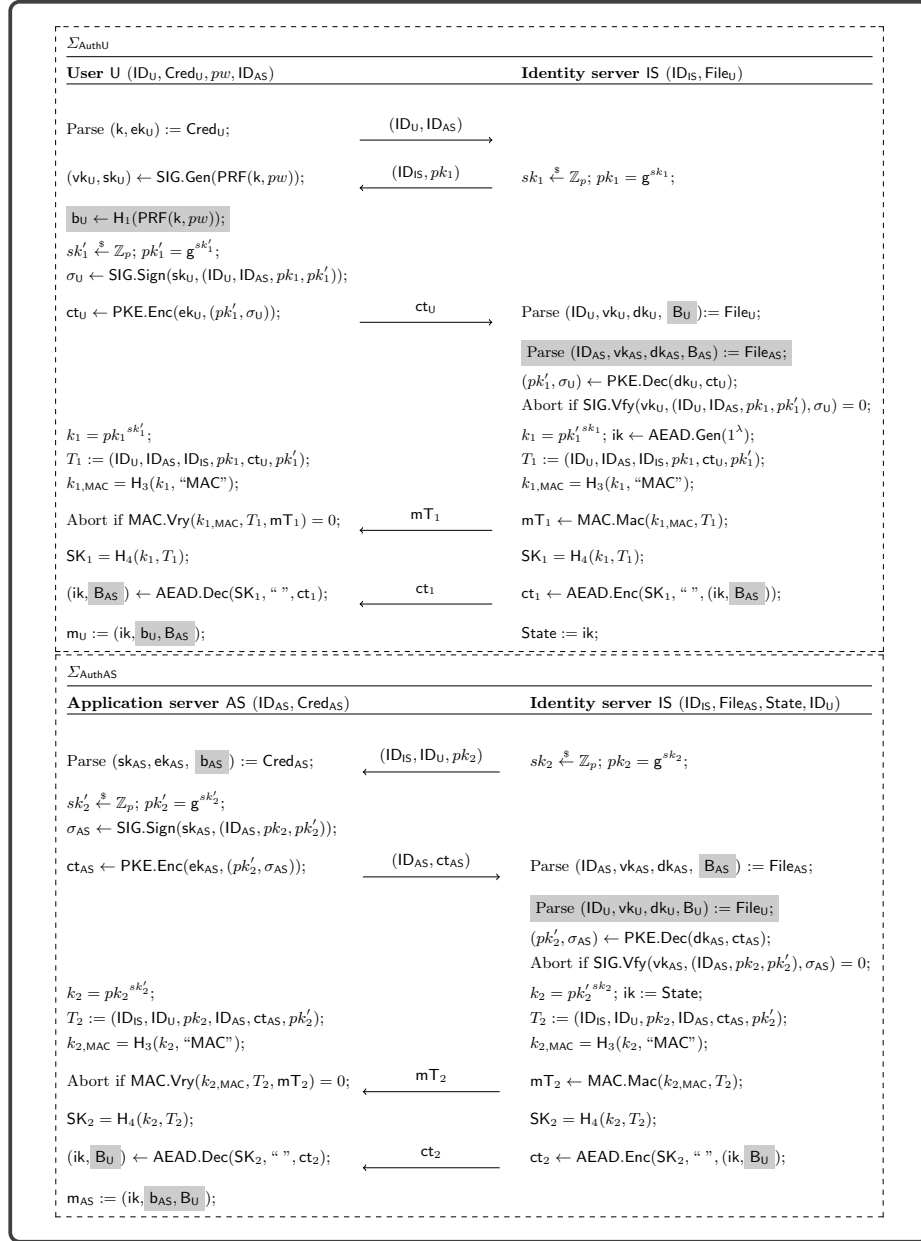
## 5 Correctness and Security Analysis

*Security properties.* An SaPBC protocol requires to provide correctness, key indistinguishability, and password recovery resistance. In this work, we also consider explicit authentication as an additional requirement for SaPBC.

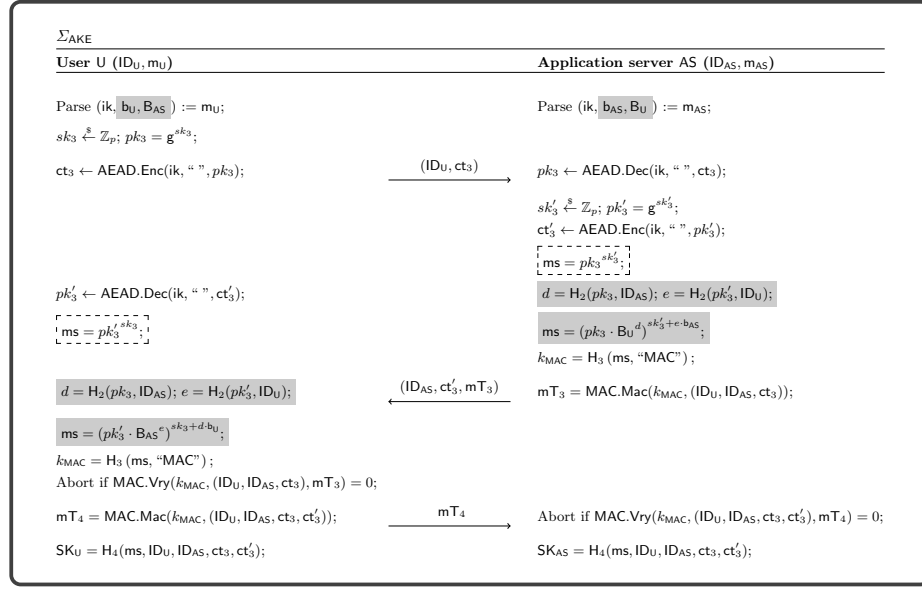
We prove that basic/enhanced SaPBC fulfills the above properties. The proofs of Theorem 2 to 7 can be found in Appendixes C, D, E, F, G, and H. Note that the sub-protocol  $\Sigma_{\text{AuthU}}$  or  $\Sigma_{\text{AuthAS}}$  of one SaPBC protocol  $\Sigma$  is essentially an authenticated and confidential channel establishment (ACCE) protocol [24], which is constructed with an AKE protocol and AEAD. Therefore, it is sufficient to prove that the AKE protocol has key indistinguishability.

We note that a “multi-stage” model was proposed in [18] for handling complex protocols. Although our existing security analysis provides an intuition that the proposed sub-protocols are secure under composition, we leave the formal analysis under the multi-stage model as future work.

**Theorem 1 (Correctness).** *The proposed basic SaPBC protocol  $\Sigma^B$  and the enhanced SaPBC protocol  $\Sigma^E$  satisfy correctness.*



**Fig. 5.** Authentication phase of the SaPBC protocol  $\Sigma$  ( $\Sigma = \Sigma^B$  or  $\Sigma^E$ ). The code in gray boxes is only executed in  $\Sigma^E$ .



**Fig. 6.** AKE phase of the SaPBC protocol  $\Sigma$  ( $\Sigma = \Sigma^B$  or  $\Sigma^E$ ). The code in dashed boxes is only executed in  $\Sigma^B$ , and the code in gray boxes is only executed in  $\Sigma^E$ .

*Proof.* The correctness is easily verified. In the protocol  $\Sigma$  ( $\Sigma = \Sigma^B$  or  $\Sigma^E$ ), a user  $U$  with the correct password  $pw$  and key  $k$  can recover the signature private key  $sk_U$ . With  $sk_U$ , the signature  $\sigma_U$  is generated and then encrypted using the encryption public key  $ek_U$ . The identity server  $IS$  will decrypt the ciphertext and check the validity of  $\sigma_U$ . If  $\sigma_U$  is valid,  $IS$  is assured that  $U$  is legitimate, and then negotiates a shared key  $k_1$ . With  $k_1$ ,  $IS$  derives a session key  $SK_1$  to securely issue  $ik$  or  $(ik, B_{AS})$ , such that  $U$  can obtain the shared message  $m_U$ . Similarly,  $IS$  checks legality of the application server  $AS$  by verifying it holds the correct  $ek_{AS}$  and  $sk_{AS}$ . If  $AS$  passes the check,  $AS$  will obtain the shared message  $m_{AS}$ . Using the shared messages,  $U$  and  $AS$  can establish the shared secret  $ms$ , authenticate each other's identity via verification of  $MAC$ , and obtain the shared session key  $SK_U = SK_{AS} = H_4(ms, ID_U, ID_{AS}, ct_3, ct'_3)$ , where

$$\begin{aligned}
 ms &= pk_3^{sk'_3} = (g^{sk'_3})^{sk_3} = (g^{sk_3})^{sk'_3} = pk_3^{sk'_3}, & \text{if } \Sigma = \Sigma^B; \\
 ms &= (pk'_3 \cdot B_{AS}^e)^{sk_3 + d \cdot b_U} = (g^{sk'_3 + b_{AS} \cdot e})^{(sk_3 + d \cdot b_U)} \\
 &= (g^{sk_3 + d \cdot b_U})^{(sk'_3 + b_{AS} \cdot e)} = (pk_3 \cdot B_U^d)^{sk'_3 + e \cdot b_{AS}}, & \text{if } \Sigma = \Sigma^E.
 \end{aligned} \tag{4}$$

□

**Theorem 2 (Password Recovery Resistance,  $\Sigma_{RegU}^B$ ).** Assuming confidential and authenticated channels, the registration subprotocol  $\Sigma_{RegU}^B$  (Fig. 4) ex-

cuted between  $\mathcal{U}$  and  $\mathcal{IS}$  in the basic SaPBC protocol achieves resistance against password-recovery in the sense of Definition 5. More specifically, if PRF is secure and modeled as a random oracle, it holds for the advantage of any PPT adversary  $\mathcal{A}$  with running time  $t$  against  $\Sigma_{\text{RegU}}^{\text{B}}$  in the random oracle model that

$$\text{Adv}_{\mathcal{A}, \Sigma_{\text{RegU}}^{\text{B}}}^{\text{pw}} \leq d \cdot (\epsilon_{\text{PRF}} + 2^{-\delta+1} + 2^\lambda), \quad (5)$$

where  $\delta$  is the minimal entropy of  $\text{pw}$  and  $\epsilon_{\text{PRF}}$  is the advantage of any adversary with running time  $t' \approx t$  against PRF.

**Theorem 3 (Key Indistinguishability,  $\Sigma_{\text{AuthU}}^{\text{B}}$ ).** *The subprotocol  $\Sigma_{\text{AuthU}}^{\text{B}}$  (Fig. 5) in the basic SaPBC protocol has key indistinguishability in the sense of Definition 6, provided that PRF is secure and modeled as a random oracle, SIG is EUF-CMA secure, MAC is EUF-CMA secure, and GCDH is hard w.r.t.  $\mathbb{G}$ . For every PPT adversary  $\mathcal{A}$  with running time  $t$ , it holds that*

$$\text{Adv}_{\mathcal{A}, \Sigma_{\text{AuthU}}^{\text{B}}}^{\text{key}} \leq \frac{(q_{\text{Send}})^2 + q_{\text{H}}^2}{2^{L_{\text{H}}}} + (d\ell)^2 \left( \frac{1}{2^\lambda} + \frac{1}{2^\delta} + \epsilon_{\text{SIG}} + \text{Adv}_{\mathcal{B}_1}^{\text{GCDH}} + \epsilon_{\text{MAC}} + \text{Adv}_{\mathcal{B}_2}^{\text{GCDH}} \right), \quad (6)$$

where  $\delta$  is the minimal entropy of  $\text{pw}$ ,  $q_{\text{Send}}$  is the number of  $\text{Send}()$  queries,  $q_{\text{H}}$  is the number of hash queries,  $L_{\text{H}}$  is the minimum size of hash queries' outputs and partial transcripts,  $\mathcal{B}_1$  and  $\mathcal{B}_2$  are GCDH-adversaries,  $\epsilon_{\text{SIG}}$  is the advantage of any adversary against SIG,  $\epsilon_{\text{MAC}}$  is the advantage of any adversary against MAC, and all these adversaries are with running time upper bounded by  $t' \approx t$ .

**Theorem 4 (Key Indistinguishability,  $\Sigma_{\text{AKE}}^{\text{B}}$ ).** *The subprotocol  $\Sigma_{\text{AKE}}^{\text{B}}$  (Fig. 6) in the basic SaPBC protocol has key indistinguishability in the random oracle model in the sense of Definition 6, provided that AEAD is secure, MAC is EUF-CMA secure, and GCDH is hard w.r.t.  $\mathbb{G}$ . More specifically, it holds for every PPT adversary  $\mathcal{A}$  with running time  $t$  that*

$$\begin{aligned} \text{Adv}_{\mathcal{A}, \Sigma_{\text{AKE}}^{\text{B}}}^{\text{key}} &\leq \frac{(q_{\text{Send}})^2 + q_{\text{H}}^2}{2^{L_{\text{H}}}} + (d\ell)^2 (\text{Adv}_{\mathcal{D}_1, \Sigma_{\text{AuthU}}^{\text{B}}}^{\text{key}} + \text{Adv}_{\mathcal{D}_2, \Sigma_{\text{AuthAS}}^{\text{B}}}^{\text{key}} \\ &\quad + \epsilon_{\text{AEAD}} + \text{Adv}_{\mathcal{B}_1}^{\text{GCDH}} + \epsilon_{\text{MAC}} + \text{Adv}_{\mathcal{B}_2}^{\text{GCDH}}), \end{aligned} \quad (7)$$

where  $\epsilon_{\text{AEAD}}$  is the advantage of any adversary with running time  $t' \approx t$  against AEAD,  $\mathcal{D}_1$  is an AKE-adversary against  $\Sigma_{\text{AuthU}}^{\text{B}}$ ,  $\mathcal{D}_2$  is an AKE-adversary against  $\Sigma_{\text{AuthAS}}^{\text{B}}$  and its advantage  $\text{Adv}_{\mathcal{D}_2, \Sigma_{\text{AuthAS}}^{\text{B}}}^{\text{key}}$  can be obtained in a similar way to getting  $\text{Adv}_{\mathcal{D}_1, \Sigma_{\text{AuthU}}^{\text{B}}}^{\text{key}}$ , and other notations are defined as above.

**Theorem 5 (Password Recovery Resistance,  $\Sigma_{\text{AuthU}}^{\text{B}}$  and  $\Sigma_{\text{AKE}}^{\text{B}}$ ).** *Assuming confidential and authenticated channels for initialization, the subprotocols  $\Sigma_{\text{AuthU}}^{\text{B}}$  and  $\Sigma_{\text{AKE}}^{\text{B}}$  (Figs. 5 and 6) in the basic SaPBC protocol, if  $\mathcal{IS}$  faithfully binds the  $(\mathcal{U}, \mathcal{AS})$  relation in honest authentication, have resistance against password-recovery in the sense of Definition 5. More specifically, if PRF is secure, PKE is IND-CCA secure, AEAD is secure, and GCDH is hard w.r.t.  $\mathbb{G}$ , it*

holds for the advantage of any adversary  $\mathcal{A}$  with running time  $t$  against  $\Sigma_{\text{AuthU}}^{\text{B}}$  and  $\Sigma_{\text{AKE}}^{\text{B}}$  in the random oracle model that

$$\begin{aligned} \text{Adv}_{\mathcal{A}, \Sigma_{\text{AuthU}}^{\text{B}}, \Sigma_{\text{AKE}}^{\text{B}}}^{\text{pw}} &\leq \frac{(d\ell)^2}{2^{L_H}} + \epsilon_{\text{PRF}} + (d\ell)^2 (\text{Adv}_{\mathcal{D}_1, \Sigma_{\text{AuthU}}^{\text{B}}}^{\text{key}} \\ &\quad + \epsilon_{\text{PKE}} + \epsilon_{\text{AEAD}} + \text{Adv}_{\mathcal{D}_2, \Sigma_{\text{AKE}}^{\text{B}}}^{\text{key}} + 2^{-\delta}), \end{aligned} \quad (8)$$

where  $\mathcal{D}_1$  and  $\mathcal{D}_2$  are AKE-adversaries against  $\Sigma_{\text{AuthU}}^{\text{B}}$  and  $\Sigma_{\text{AKE}}^{\text{B}}$  respectively,  $\epsilon_{\text{PKE}}$  is the advantage of any adversary with running time  $t' \approx t$  against PKE, and other notations are defined as above.

**Theorem 6 (Authentication,  $\Sigma_{\text{AuthU}}^{\text{B}}$ ).** *The subprotocol  $\Sigma_{\text{AuthU}}^{\text{B}}$  (Fig. 5) in the basic SaPBC has authentication in the sense of Definition 7, provided that PRF is secure, SIG is EUF-CMA secure, MAC is EUF-CMA secure, PKE is IND-CCA secure, and GCDH is hard w.r.t.  $\mathbb{G}$ . For any adversary  $\mathcal{A}$  with running time  $t$ , the advantage  $\text{Adv}_{\mathcal{A}, \Sigma_{\text{AuthU}}^{\text{B}}}^{\text{auth}}$  that  $\mathcal{A}$  makes an oracle accept maliciously satisfies*

$$\text{Adv}_{\mathcal{A}, \Sigma_{\text{AuthU}}^{\text{B}}}^{\text{auth}} \leq \frac{(q_{\text{Send}})^2 + q_{\text{H}}^2}{2^{L_H-1}} + \epsilon_{\text{PRF}} + d\ell(2\epsilon_{\text{PKE}} + \frac{q_{\text{Send}}}{p} + \text{Adv}_{\mathcal{B}_1}^{\text{GCDH}} + \epsilon_{\text{MAC}} + \epsilon_{\text{SIG}}), \quad (9)$$

where all notations are defined as above.

Compared to  $\Sigma_{\text{AuthU}}^{\text{B}}$ , the subprotocol  $\Sigma_{\text{AuthAS}}^{\text{B}}$  (Fig. 5) in the basic SaPBC protocol also has authentication in the sense of Definition 7. The theorem and the proof are almost identical to those of Theorem 6. The difference is that the parts involving PRF and  $\text{pw}$  are not needed. Hence we would omit them here.

Next, Theorem 7 shows that the subprotocol  $\Sigma_{\text{AKE}}^{\text{B}}$  has authentication.

**Theorem 7 (Authentication,  $\Sigma_{\text{AKE}}^{\text{B}}$ ).** *The subprotocol  $\Sigma_{\text{AKE}}^{\text{B}}$  (Fig. 6) in the basic SaPBC protocol has authentication in the sense of Definition 7, provided that AEAD is secure, MAC is EUF-CMA secure, and GCDH is hard w.r.t.  $\mathbb{G}$ . For any adversary  $\mathcal{A}$  with running time  $t$ , the advantage  $\text{Adv}_{\mathcal{A}, \Sigma_{\text{AKE}}^{\text{B}}}^{\text{auth}}$  that  $\mathcal{A}$  makes an oracle accept maliciously satisfies*

$$\begin{aligned} \text{Adv}_{\mathcal{A}, \Sigma_{\text{AKE}}^{\text{B}}}^{\text{auth}} &\leq \frac{(q_{\text{Send}})^2 + q_{\text{H}}^2}{2^{L_H-1}} + 2(d\ell)(\text{Adv}_{\mathcal{D}_1, \Sigma_{\text{AuthU}}^{\text{B}}}^{\text{key}} + \text{Adv}_{\mathcal{D}_2, \Sigma_{\text{AuthAS}}^{\text{B}}}^{\text{key}} \\ &\quad + \epsilon_{\text{AEAD}} + \text{Adv}_{\mathcal{B}_1}^{\text{GCDH}} + \epsilon_{\text{MAC}}), \end{aligned} \quad (10)$$

where all notations are defined as above.

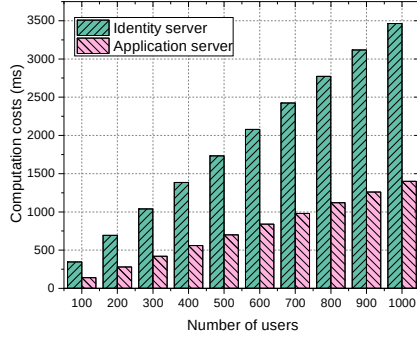
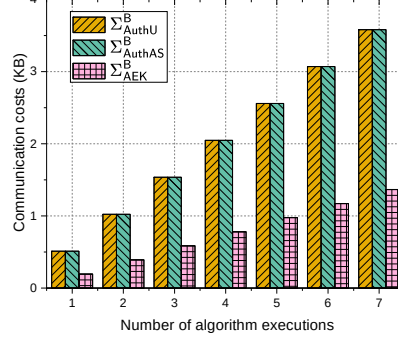
**Security of the Enhanced SaPBC Protocol.** The enhanced subprotocol  $\Sigma_{\text{RegU}}^{\text{E}}$  enjoys almost the same security of the basic one  $\Sigma_{\text{RegU}}^{\text{B}}$ . The key indistinguishability and authentication of  $\Sigma_{\text{AuthU}}^{\text{E}}$  (resp.  $\Sigma_{\text{AuthAS}}^{\text{E}}$ ) is the same as  $\Sigma_{\text{AuthU}}^{\text{B}}$  (resp.  $\Sigma_{\text{AuthAS}}^{\text{B}}$ ).

The merit of the enhancement lies in the strong security of the AKE phase. More specifically,  $\Sigma_{\text{AKE}}^{\text{E}}$  can additionally tolerate `EphemeralKeyReveal()` in non-trivial cases in the AKE phase, where `EphemeralKeyReveal()` is the query that



**Table 4.** Computation costs of each entity in different algorithms (ms).

|                       | Basic SaPBC $\Sigma^B$ |                     |                  | Enhanced SaPBC $\Sigma^E$ |                     |                  |
|-----------------------|------------------------|---------------------|------------------|---------------------------|---------------------|------------------|
|                       | $\Sigma_{AuthU}^B$     | $\Sigma_{AuthAS}^B$ | $\Sigma_{AKE}^B$ | $\Sigma_{AuthU}^E$        | $\Sigma_{AuthAS}^E$ | $\Sigma_{AKE}^E$ |
| User U                | 0.841                  | -                   | 0.627            | 0.867                     | -                   | 0.892            |
| Identity server IS    | 1.733                  | 1.731               | -                | 1.733                     | 1.731               | -                |
| Application server AS | -                      | 0.772               | 0.627            | -                         | 0.772               | 0.892            |

**Fig. 7.** Computation costs of the identity server and the application server different number of users.**Fig. 8.** Communication costs with different number of algorithm executions for the basic SaPBC protocol.

leaks the ephemeral randomness, such as  $sk_1$ ,  $sk_2$  or  $ik$ . In  $\Sigma_{AKE}^E$ , even if these ephemeral secrets are leaked to  $\mathcal{A}$ , it cannot compute the session key. Moreover, a leakage of the session key in the related  $\Sigma_{AuthU}^E$  via `RevealKey()` allows  $\mathcal{A}$  sees the public DH-shares, but  $\mathcal{A}$  cannot compute the session key in  $\Sigma_{AKE}^E$ .

For password-recovery, an extra trivial case has to be excluded for  $\Sigma_{AuthU}^E$  and  $\Sigma_{AKE}^E$ : `LeakDK(U)/Corrupt(U)` and `Corrupt(AS)` for partnered U-AS sessions.

## 6 Performance Evaluation

We implement the basic/enhanced SaPBC protocol using Java language and Bouncy Castle library<sup>3</sup>. In the implementation, we set the security level to be 128 bits. SIG is implemented using ECDSA over the elliptic curve *secp256r1*, the Diffie-Hellman key agreement is instantiated with the elliptic curve group constructed by *secp256r1*, PKE is implemented via RSA with the 3072-bit modulus, AEAD is implemented using AES-CRT with 128-bit keys and SHA-256, and PRF, MAC, and hash functions are instantiated with SHA-256. Each entity's identity is set as 20 characters. We conduct experiments on a desktop with Ubuntu system, an Intel Core i7 CPU, and 32 GB DDR3 of RAM. To obtain reliable experimental results, we execute 1000 trials for each experiment.

<sup>3</sup> Bouncy Castle is an open-source cryptographic library.

*Computation Overhead.* We provide the theoretical analysis of computation overheads in Appendix I. Table 4 shows the computation costs of entities in the phases of authentication and AKE. We can see that one authentication takes less than 2 ms for U, demonstrating practicality of our basic/enhanced SaPBC protocols. The computation overheads of IS authenticating U and AS are almost identical, and U takes the same time as AS in the AKE phase. We evaluate the computation costs of IS and AS with different number of users. As illustrated in Fig. 7, the computation cost of IS (resp. AS) is around 3500 ms (resp. 1400 ms) when interacting with 1000 users simultaneously.

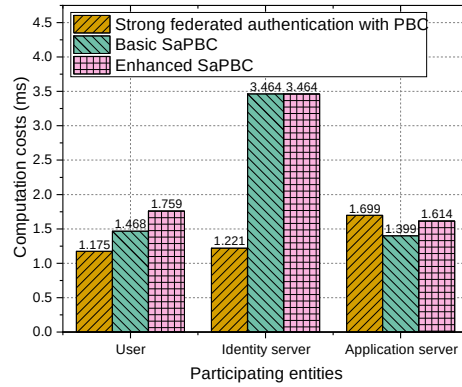
*Communication Overhead.* In the basic SaPBC protocol  $\Sigma^B$ , the communication overheads on U and IS in  $\Sigma_{AuthU}^B$  are composed of 3 identities, a PKE ciphertext, a group element, a secret key  $ik$ , and a tag. Let  $|id|$  denote the identity size,  $|\mathbb{G}|$  the element length in  $\mathbb{G}$ ,  $|c|$  the size of one PKE ciphertext,  $|ik|$  the size of  $ik$ , and  $|mT|$  the size of a tag. The communication overheads are  $3|id| + |c| + |\mathbb{G}| + |ik| + |mT|$ . In  $\Sigma_{AuthAS}^B$ , the communication costs are identical to those in  $\Sigma_{AuthU}^B$ . Furthermore, we can get the communication overheads of U or AS in  $\Sigma_{AKE}^B$  are  $2|id| + 2|\mathbb{G}| + 2|mT|$ . Compared with  $\Sigma^B$ , the enhanced SaPBC protocol  $\Sigma^E$  requires extra transmission of one group element in each authentication. Fig. 8 depicts the experimental performance evaluation  $\Sigma^B$ . The whole communication costs for one authentication are 1.219 KB.

*Comparison with Existing Schemes.* We conduct experiments to perform a performance comparison between our basic/enhanced SaPBC and strong federated authentication with PBC [36]. For one authentication, the total communication costs in basic/enhanced SaPBC are about 1.2 KB, which are higher than those (0.41 KB) in strong federated authentication with PBC. This is because basic/enhanced SaPBC needs additional authentication of AS and negotiation of session keys. Fig. 9 provides the comparison of different entities' computation costs. U's computation cost in basic/enhanced SaPBC is slightly higher than that in strong federated authentication with PBC due to the extra session key agreement. For IS, basic/enhanced SaPBC requires it to additionally authenticate AS, and hence more costs are incurred. Fig. 9 also indicates that the two SaPBC protocols outperform the existing scheme in terms of AS's computation overhead, as costly verification of signature proofs of knowledge is not needed.

*Limitations.* In  $\Sigma^B$  and  $\Sigma^E$ , the key  $k$  is stored in U's device. Only if U inputs  $pw$  into this device, the signature private key  $sk_U$  can be recovered for authentication. Therefore, in the setting of multiple devices, U has to perform registration for each device. Moreover, as discussed in Section 5, once an adversary  $\mathcal{A}$  compromises  $k$  and  $B_U$  at the AS end in  $\Sigma^E$ ,  $\mathcal{A}$  could guess  $pw$  offline.

## 7 Conclusion

In this paper, we have proposed strong federated authentication with password-based credential against identity server corruption (SaPBC), and defined a for-



**Fig. 9.** Computation overhead comparison.

mal security model with fine-grained information leakage. Under the security model, two SaPBC protocols have been presented based on standard building blocks to achieve different security levels. We have provided formal security proofs, which demonstrate that the two protocols satisfy the properties of correctness, key indistinguishability, password recovery resistance, and authentication. Theoretical and experimental performance evaluation show that the presented protocols are efficient in terms of computation and communication.

**Acknowledgment.** This work was supported in part by the National Nature Science Foundation of China under Grant 62272091, in part by the Stability Program of Science and Technology on Communication Security Laboratory (2023), in part by the Lee Kong Chian Fellowship awarded by Singapore Management University, and in part by the National Natural Science Foundation of China under Grant 62202197.

## References

1. What is OpenID Connect. <https://openid.net/developers/how-connect-works/>. Accessed on September 10, 2024
2. The OAuth 2.0 Authorization Framework: Bearer Token Usage. <https://datatracker.ietf.org/doc/html/rfc6750>. Accessed on August 21, 2024
3. Abdalla, M., Pointcheval, D.: Simple password-based encrypted key exchange protocols. In: Proc. CT-RSA. pp. 191–208 (2005)
4. Agrawal, S., Miao, P., Mohassel, P., Mukherjee, P.: PASTA: password-based threshold authentication. In: Proc. ACM CCS. pp. 2042–2059 (2018)
5. Alwen, J., Coretti, S., Dodis, Y.: The double ratchet: security notions, proofs, and modularization for the signal protocol. In: Proc. EUROCRYPT. pp. 129–158 (2019)
6. Armando, A., Carbone, R., Compagna, L., Cuellar, J., Tobarra, L.: Formal analysis of SAML 2.0 web browser single sign-on: breaking the SAML-based single sign-on for google apps. In: Proc. FMSE. pp. 1–10 (2008)

7. Barbosa, M., Boldyreva, A., Chen, S., Warinschi, B.: Provable security analysis of FIDO2. In: Proc. CRYPTO. pp. 125–156 (2021)
8. Bartusek, J., Garg, S., Jain, A., Policharla, G.V.: End-to-end secure messaging with traceability only for illegal content. In: Proc. EUROCRYPT. pp. 35–66 (2023)
9. Bellare, M., Pointcheval, D., Rogaway, P.: Authenticated key exchange secure against dictionary attacks. In: Proc. EUROCRYPT. pp. 139–155 (2000)
10. Bindel, N., Cremers, C., Zhao, M.: Fido2, ctap 2.1, and webauthn 2: Provable security and post-quantum instantiation. In: Proc. IEEE Security&Privacy. pp. 1471–1490 (2023)
11. Boeyen, S., Santesson, S., Polk, T., Housley, R., Farrell, S., Cooper, D.: Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile. RFC 5280 (May 2008)
12. Boyle, E., Goldwasser, S., Ivan, I.: Functional signatures and pseudorandom functions. In: Proc. PKC. pp. 501–519 (2014)
13. Chakraborty, D., Bugiel, S.: SimFIDO: FIDO2 user authentication with simtpm. In: Proc. ACM CCS. pp. 2569–2571 (2019)
14. Davit, B., Roni, S., Brad, H., Jeff, H., Ka, Y.: FIDO UAF Authenticator Commands. <https://fidoalliance.org/specs/fido-uaf-v1.2-ps-20201020/fido-uaf-authnr-cmds-v1.2-ps-20201020.html>. Accessed on September 10, 2024
15. Davit, B., Roni, S., Brad, H., Jeff, H., Ka, Y.: FIDO UAF Authenticator Commands. FIDO Alliance (2020)
16. Dayanikli, D., Lehmann, A.: Password-based credentials with security against server compromise. In: Proc. ESORICS. pp. 147–167 (2023)
17. Diffie, W., Hellman, M.E.: New directions in cryptography. *IEEE Transactions on Information Theory* **22**(6) (1976)
18. Dowling, B., Fischlin, M., Günther, F., Stebila, D.: A cryptographic analysis of the tls 1.3 handshake protocol. *Journal of Cryptology* **34**(4), 37 (2021)
19. Faller, S., Handirk, T., Hesse, J., Horváth, M., Lehmann, A.: Password-protected key retrieval with (out) hsm protection. *Proc. ACM CCS* (2024)
20. Gibbs, S.: Dropbox hack leads to leaking of 68m user passwords on the internet (2016), <https://www.theguardian.com/technology/2016/aug/31/dropbox-hack-passwords-68m-data-breach>. Accessed on September 10, 2024
21. Goel, V., Perlroth, N.: Yahoo says 1 billion user accounts were hacked (2016), <https://www.nytimes.com/2016/12/14/technology/yahoo-hack.html>. Accessed on September 10, 2024
22. Hackett, R.: LinkedIn lost 167 million account credentials in data breach (2016), <https://fortune.com/2016/05/18/linkedin-data-breach-email-password/>. Accessed on September 10, 2024
23. Hanzlik, L., Loss, J., Wagner, B.: Token meets wallet: Formalizing privacy and revocation for fido2. In: Proc. IEEE Security&Privacy. pp. 1491–1508 (2023)
24. Jager, T., Kohlar, F., Schäge, S., Schwenk, J.: On the security of tls-dhe in the standard model. In: Proc. CRYPTO. pp. 273–293 (2012)
25. Jiang, C., Xu, C., Han, Y., Zhang, Z., Chen, K.: Two-factor authenticated key exchange from biometrics with low entropy rates. *IEEE Trans. on Inf. Forensics and Security* **19**, 3844–3856 (2024)
26. Jiang, C., Xu, C., Yang, G., Zhang, Z., Chen, J.: Device-enhanced password-based threshold single-sign-on authentication. *IEEE Trans. on Inf. Forensics and Security* **20**, 2006–2021 (2025)
27. Jiang, J., Wang, D., Zhang, G., Chen, Z.: Quantum-resistant password-based threshold single-sign-on authentication with updatable server private key. In: Proc. ESORICS. pp. 295–316 (2022)

28. Kitagawa, F., Morimae, T., Nishimaki, R., Yamakawa, T.: Quantum public-key encryption with tamper-resilient public keys from one-way functions. In: Proc. CRYPTO. pp. 93–125 (2024)
29. Kuchhal, D., Saad, M., Oest, A., Li, F.: Evaluating the security posture of real-world fido2 deployments. In: Proc. ACM CCS. pp. 2381–2395 (2023)
30. Li, Y., Schäge, S.: No-match attacks and robust partnering definitions: defining trivial attacks for security protocols is not trivial. In: Proc. ACM CCS. pp. 1343–1360 (2017)
31. Menezes, A.J., Van Oorschot, P.C., Vanstone, S.A.: Handbook of applied cryptography. CRC press (2018)
32. Rescorla, E.: The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446 (Aug 2018)
33. Reynolds, J., Smith, T., Reese, K., Dickinson, L., Ruoti, S., Seamons, K.: A tale of two studies: The best and worst of yubikey usability. In: Proc. IEEE Security & Privacy. pp. 872–888 (2018)
34. Shoup, V.: Sequences of games: a tool for taming complexity in security proofs. cryptology eprint archive (2004)
35. Xu, R., Yang, S., Zhang, F., Fang, Z.: Miso: legacy-compatible privacy-preserving single sign-on using trusted execution environments. In: Proc. EuroS&P. pp. 352–372 (2023)
36. Zhang, Z., Wang, Y., Yang, K.: Strong authentication without temper-resistant hardware and application to federated identities. In: Proc. NDSS (2020)

## A Related Work

*Security Models.* Existential Unforgeability under Chosen Message and Chosen Verification Queries Attack (EUF-CMVA) [36] was defined for PBC. Consider adversaries' capability in practice, they are allowed to access oracles **RevealCred** and **RevealPw**, which captures leakage of password-protected credentials and that of passwords, respectively. However, this model does not consider multiple sessions and always assume authenticated and confidential channels exist between parties. Inspired by the security models for password-based authenticated key exchange (PAKE) protocols [3,9] and password-based data retrieval [19], we formalize the SaPBC security notion with *key indistinguishability* in a multi-party, multi-session setting. This further ensures that any adversary cannot distinguish a session key generated in an honest execution of SaPBC from a random value, covering another possible consequence of online password guesses.

*Federated Identities.* In a federated identity system, a user can sign on using a standard mechanism, e.g., username/password verification, and obtain a token to access diverse services that different application servers provide. However, once an adversary  $\mathcal{A}$  compromises a password-related file (e.g., the password's hash),  $\mathcal{A}$  could launch offline password guesses and further impersonate the user. To address this issue, recent research employs multiple identity servers to identify a user in a threshold way [4,26,27], which disables offline password guesses.

*Strong Authentication.* Strong authentication enables a user to authenticate herself/himself by proving knowledge of a secret via a cryptographic challenge-response mechanism [29]. Fast Identity Online (FIDO) Alliance, an open industry association, has published strong authentication specifications including Universal Authentication Framework (UAF) [14] and FIDO2 [7,13]. Barbosa et al. [7] provided the first formal security model and analysis for FIDO2 protocols including W3C's Web Authentication (WebAuthn) and the Client-to-Authenticator Protocol v2.0 (CTAP2). Afterwards, Bindel et al. [10] extended the initial security analysis of FIDO2 to consider more properties, e.g., token binding agreement and user verification. Hanzlik et al. [23] adapt Barbosa et al.'s security model to capture authentication tokens with key derivation or key wrapping, and addressed the global key revocation problem. Nevertheless, strong authentication protocols including FIDO2 requires tamper-resistant hardware to protect the secret at user end. Our work is able to support strong authentication without relying on tamper-resistant hardware.

## B Cryptographic Primitives

**Pseudorandom function** [12]. A pseudorandom function  $\text{PRF} : \mathcal{K} \times \{0, 1\}^n \rightarrow \mathcal{R}$  with key space  $\mathcal{K}$ , domain  $\{0, 1\}^n$ , and range  $\mathcal{R}$  is secure if for any PPT adversary  $\mathcal{A}$ , it holds that for  $k \xleftarrow{\$} \mathcal{K}$  and  $f \xleftarrow{\$} \text{Func}[\{0, 1\}^n, \mathcal{R}]$ ,

$$\left| \Pr[\mathcal{A}^{\text{PRF}(k, \cdot)}(1^\lambda) = 1] - \Pr[\mathcal{A}^{f(\cdot)}(1^\lambda) = 1] \right| \leq \epsilon(\lambda).$$

**Digital signature** [8]. A digital signature scheme  $\text{SIG}$  consists of three algorithms  $\{\text{SIG.Gen}, \text{SIG.Sign}, \text{SIG.Vfy}\}$ .

- $(\text{vk}, \text{sk}) \leftarrow \text{SIG.Gen}(1^\lambda)$ . The key generation algorithm takes as input  $1^\lambda$ , and outputs a public key  $\text{vk}$  and a private key  $\text{sk}$ .
- $\sigma \leftarrow \text{SIG.Sign}(\text{sk}, \text{m})$ . The signing algorithm takes a private key  $\text{sk}$  and a message  $\text{m}$  as input, and outputs a signature  $\sigma$ .
- $0/1 \leftarrow \text{SIG.Vfy}(\text{vk}, \text{m}, \sigma)$ . The verification algorithm takes as input a public key  $\text{vk}$ , a message  $\text{m}$ , and a signature  $\sigma$ , and outputs 1 if  $\sigma$  is a valid signature of  $\text{m}$  under  $\text{sk}$ , and 0 otherwise.

The key generation algorithm may be represented as  $(\text{vk}, \text{sk}) \leftarrow \text{SIG.Gen}(r)$ , which behaves deterministically when inputting an explicit random number  $r \in \{0, 1\}^\lambda$ .  $\text{SIG}$  is existentially unforgeable under chosen message attack (EUF-CMA) if for any PPT adversary  $\mathcal{A}$ ,

$$\Pr \left[ \begin{array}{c} \text{m} \notin Q \wedge \\ \text{SIG.Vfy}(\text{vk}, \text{m}, \sigma) = 1 \end{array} : \begin{array}{c} (\text{vk}, \text{sk}) \leftarrow \text{SIG.Gen}(1^\lambda) \\ (\text{m}, \sigma) \leftarrow \mathcal{A}^{\text{SIG.Sign}(\text{sk}, \cdot)}(\text{vk}) \end{array} \right] = \epsilon(\lambda),$$

where  $Q$  is the set of message queries that  $\mathcal{A}$  makes to  $\text{SIG.Sign}(\text{sk}, \cdot)$ .

**Public-key encryption** [28]. A public-key encryption scheme  $\text{PKE}$  consists of three algorithms  $\{\text{PKE.Gen}, \text{PKE.Enc}, \text{PKE.Dec}\}$ .

- $(\text{ek}, \text{dk}) \leftarrow \text{PKE.Gen}(1^\lambda)$ . The key generation algorithm takes as input  $1^\lambda$  and outputs a public key  $\text{ek}$  and a private key  $\text{dk}$ .
- $\text{ct} \leftarrow \text{PKE.Enc}(\text{ek}, \text{m})$ . The encryption algorithm takes as input a public key  $\text{ek}$  and a message  $\text{m}$ , and outputs a ciphertext  $\text{ct}$ .
- $\text{m} \leftarrow \text{PKE.Dec}(\text{dk}, \text{ct})$ . The decryption algorithm takes as input a private key  $\text{dk}$  and a ciphertext  $\text{ct}$ , and outputs a message  $\text{m}$ .

$\text{PKE}$  is required to achieve indistinguishability under chosen ciphertext attack (IND-CCA) [28].

**Authenticated encryption with associated data** [5]. An authenticated encryption with associated data scheme AEAD is a tuple of algorithms  $\{\text{AEAD.Gen}, \text{AEAD.Enc}, \text{AEAD.Dec}\}$ .

- $K \leftarrow \text{AEAD.Gen}(1^\lambda)$ . The key generation algorithm takes as input  $1^\lambda$  and outputs a secret key  $K$ .
- $\text{ct} \leftarrow \text{AEAD.Enc}(K, \text{a}, \text{m})$ . The encryption algorithm takes a secret key  $K$ , associate data  $\text{a}$ , and a message  $\text{m}$  as input, and outputs a ciphertext  $\text{ct}$ .
- $\text{m} \leftarrow \text{AEAD.Dec}(K, \text{a}, \text{ct})$ . The decryption algorithm takes as input a secret key  $K$ , associate data  $\text{a}$ , and a ciphertext  $\text{ct}$ , and outputs a message  $\text{m}$ .

The correctness of AEAD requires that decryption of  $\text{ct}$  returns  $\text{m}$  if the same  $K$  and  $\text{a}$  are input, and  $\perp$  otherwise. Besides IND-CCA, AEAD also ensures message authentication [5].

**Message authenticated code** [30]. A message authenticated code scheme MAC consists of the algorithms  $\{\text{MAC.Mac}, \text{MAC.Vfy}\}$ .

- $\delta \leftarrow \text{MAC.Mac}(\mathbf{K}, \mathbf{m})$ . The tag-generation algorithm takes as input a key  $\mathbf{K}$  and a message  $\mathbf{m}$ , and outputs a tag  $\mathbf{mT}$ .
- $0/1 \leftarrow \text{MAC.Vry}(\mathbf{K}, \mathbf{m}, \mathbf{mT})$ . The verification algorithm takes as input a key  $\mathbf{K}$ , a message  $\mathbf{m}$ , and a tag  $\mathbf{mT}$ . It outputs 1 if  $\mathbf{mT}$  is a valid tage of  $\mathbf{m}$  under  $\mathbf{K}$ , and 0 otherwise.

We consider the following security game  $\mathbf{G}_{\text{MAC}}^{\text{EUF-CMA}}(\mathcal{A})$  for MAC:

1.  $\mathbf{K} \xleftarrow{\$} \mathcal{K}$
2. Run  $\mathcal{A}^{\text{MAC.Mac}(\mathbf{K}, \cdot), \text{MAC.Vry}(\mathbf{K}, \cdot)}$ .
3. If  $\mathcal{A}$  performs a verification of  $(\mathbf{m}, \mathbf{mT})$  such that the following conditions are satisfied:
  - The verification returned 1;
  - $\mathcal{A}$  did not query to  $\text{MAC.Mac}(\mathbf{K}, \cdot)$  and  $\text{MAC.Vry}(\mathbf{K}, \cdot)$  for  $(\mathbf{m}, \mathbf{mT})$ ;
4. Then return 1 else return 0.

The advantage of  $\mathcal{A}$  is defined as follows:

$$\text{Adv}_{\mathcal{A}, \text{MAC}}^{\text{EUF-CMA}} := \Pr [\mathbf{G}_{\text{MAC}}^{\text{EUF-CMA}}(\mathcal{A}) = 1]. \quad (11)$$

We say MAC is EUF-CMA secure if  $\text{Adv}_{\mathcal{A}, \text{MAC}}^{\text{EUF-CMA}}$  is negligible [30].

**Diffie-Hellman key agreement** [31]. Let  $\mathbb{G}$  be a multiplicative cyclic group of order  $p$  with generator  $\mathbf{g}$ . For  $i \in \{0, 1\}$ , the party  $\mathcal{P}_i$  chooses a uniform  $x_i \in \mathbb{Z}_p$ , computes  $h_i = \mathbf{g}^{x_i}$ , and sends  $h_i$  to  $\mathcal{P}_{1-i}$ .  $\mathcal{P}_i$  then outputs the shared key  $k = h_{1-i}^{x_i}$ .

**Gap Computational Diffie-Hellman Problem (GCDH)**. GCDH is defined as: given public parameter  $\text{params} = (\mathbb{G}, g)$  and  $(g^a, g^b)$  for  $a, b \in \mathbb{Z}_q$ , compute the element  $Z = g^{ab}$  with the help of a Decisional Diffie-Hellman Oracle  $\mathcal{O}_{\text{DDH}}(\cdot)$ , i.e.,  $\mathcal{O}_{\text{DDH}}(\cdot)$  answers whether a given quadruple  $(g, g^a, g^b, g^c)$  has  $ab \equiv c \pmod{q}$ . We say that GCDH is hard w.r.t.  $\mathbb{G}$ , if for any PPT adversary  $\mathcal{A}$ , the following quantity is negligible:

$$\text{Adv}_{\mathcal{A}, \text{GCDH}} := \Pr [Z = g^{ab} : a, b \xleftarrow{\$} \mathbb{Z}_q; Z \leftarrow \mathcal{A}^{\mathcal{O}_{\text{DDH}}(\cdot)}(\text{params}, g^a, g^b)].$$

## C Proof of Theorem 2

*Proof.* We prove this theorem with a sequence of games [34]. Let “ $G_n \Rightarrow 1$ ” be the event that  $\mathcal{A}$  makes Game  $G_n$  output 1, and let  $\text{Adv}_n = \Pr[G_n \Rightarrow 1]$  be  $\mathcal{A}$ ’s advantage in Game  $G_n$ .

**Game  $G_0$ .** This is the original game, so we have

$$\text{Adv}_{\mathcal{A}, \Sigma_{\text{RegU}}^{\text{B}}}^{\text{PW}} = \Pr[\mathbf{G}_{\mathcal{A}, \Sigma}^{\text{PW-recov}}(1^\lambda) \Rightarrow 1] = \Pr[G_0 \Rightarrow 1]. \quad (12)$$



**Game  $G_1$ .** We add an abort rule: the challenger guesses the target user, and if the guess is wrong, abort the game. This leads to

$$\Pr[G_0 \Rightarrow 1] = d \cdot \Pr[G_1 \Rightarrow 1]. \quad (13)$$

**Game  $G_2$ .** We simulate the random oracle for  $\text{PRF}()$  as follows.

- Set up an empty list  $\mathcal{L}_{\text{PRF}}$ ;
- For a random oracle query on  $(k, pw)$ , check if  $(k, pw, y)$  for some  $y$  is in  $\mathcal{L}_{\text{PRF}}$ . If so, return  $y$ . Otherwise, pick  $y \xleftarrow{\$} \mathcal{R}_{\text{PRF}}$ , save  $(k, pw, y)$  in  $\mathcal{L}_{\text{PRF}}$ , and return  $y$ .

This leads to

$$\Pr[G_1 \Rightarrow 1] \leq \Pr[G_2 \Rightarrow 1] + \epsilon_{\text{PRF}}. \quad (14)$$

**Game  $G_3$ .** We replace the  $(vk, sk)$  with  $(vk, sk) \xleftarrow{\$} \text{SIG.Gen}(r)$ ,  $r \xleftarrow{\$} \mathcal{R}$ . If  $\mathcal{A}$  has not explicitly queried the random oracle with the correct  $(k, pw)$ ,  $\mathcal{A}$  cannot notice the change. Thus, we have

$$\Pr[G_2 \Rightarrow 1] \leq \Pr[G_3 \Rightarrow 1] + 2^{-\lambda} + 2^{-\delta}. \quad (15)$$

**Game  $G_4$ .** This game is identical to  $G_3$ . The trivial cases are excluded by  $\text{TrivialU-I}dp$  and  $\text{TrivialPw}$  defined in Fig. 3. We now consider the remaining non-trivial cases.

- Case 1:  $\mathcal{A}$  has issued  $\text{LeakFile}(\text{IS})$ , but has not issued  $\text{LeakDK}(\text{U})$  or  $\text{Corrupt}(\text{U})$ , i.e.,  $\mathcal{A}$  sees  $vk_U$  and  $dk_U$ . As  $vk_U$  and  $dk_U$  are independent of  $pw$  after the changes in  $G_3$ ,  $\mathcal{A}$ 's advantage is upper bounded by  $2^{-\delta}$ .
- Case 2:  $\mathcal{A}$  has issued  $\text{LeakDK}(\text{U})$ , but has not issued  $\text{LeakFile}(\text{IS})$  or  $\text{Corrupt}(\text{IS})$ . In this case, as  $ek_U$  is independent of  $pw$ ,  $\mathcal{A}$ 's advantage is upper bounded by  $2^{-\delta}$ .
- Case 3:  $\mathcal{A}$  has never issued any  $\text{LeakDK}(\text{U})$ ,  $\text{Corrupt}(\text{U})$ ,  $\text{LeakFile}(\text{IS})$ , or  $\text{Corrupt}(\text{IS})$ . In this case, even if  $\mathcal{A}$  has broken the confidentiality of the channel and sees  $vk_U$  and  $dk_U$ ,  $\mathcal{A}$ 's advantage is upper bounded by  $2^{-\delta}$  as discussed in Case 1.

Thus, we have

$$\Pr[G_3 \Rightarrow 1] = \Pr[G_4 \Rightarrow 1]. \quad (16)$$

$$\Pr[G_4 \Rightarrow 1] \leq 2^{-\delta}. \quad (17)$$

By collecting the (in)equalities from (12) to (17), we have proved Theorem 2.  $\square$

## D Proof of Theorem 3

*Proof.* This theorem is proved via a sequence of games. Let “ $G_n \Rightarrow 1$ ” be the event that  $\mathcal{A}$  makes Game  $G_n$  output 1, and let  $\text{Adv}_n = \Pr[G_n \Rightarrow 1] - \frac{1}{2}$  be  $\mathcal{A}$ ’s advantage in Game  $G_n$ .

**Game  $G_0$ .** This game is the original game. Thus, we have

$$\text{Adv}_{\mathcal{A}, \Sigma_{\text{AuthU}}^{\text{B}}}^{\text{key}} + \frac{1}{2} = \Pr[\mathbf{G}_{\mathcal{A}, \Sigma}^{\text{key-ind}}(1^\lambda) \Rightarrow 1] = \Pr[G_0 \Rightarrow 1] = \text{Adv}_0 + \frac{1}{2}. \quad (18)$$

**Game  $G_1$ .** In this game, all hash oracles  $H_\varsigma()$  ( $\varsigma = 1, 3, 4$ ) are simulated by maintaining hash lists  $\Gamma_H$ . For a hash query  $H_\varsigma()$  on message  $m$ , if  $(\varsigma, m, v) \in \Gamma_H$ , then  $v$  is returned as the response. Otherwise, a random value  $v$  is returned and  $(\varsigma, m, v)$  is added into  $\Gamma_H$ . According to the simulation,  $G_0$  and  $G_1$  are perfectly indistinguishable:

$$\text{Adv}_0 = \text{Adv}_1. \quad (19)$$

**Game  $G_2$ .** This game is identical to Game  $G_1$ , except that we add an abort rule: if collisions among outputs of hash queries and those among partial transcripts (e.g.,  $pk_1$  and  $ct_U$ ) occur, abort the game. The probabilities of collisions can be bounded by the birthday paradox:

$$\text{Adv}_1 \leq \text{Adv}_2 + \frac{(q_{\text{Send}})^2}{2^{L_H}} + \frac{q_H^2}{2^{L_H}}. \quad (20)$$

**Game  $G_3$ .** In this game, we let the challenger guess the target session  $(\pi_i^s, \pi_j^t)$ . If the guess is wrong, the game is aborted. We can obtain

$$\text{Adv}_2 \leq (d\ell)^2 \cdot \text{Adv}_3. \quad (21)$$

**Game  $G_4$ .** In this game,  $(vk, sk)$  is replaced with  $(vk, sk) \xleftarrow{\$} \text{SIG.Gen}(r)$ ,  $r \xleftarrow{\$} \mathcal{R}$ . If  $\mathcal{A}$  has not explicitly queried the random oracle with the correct  $(k, pw)$ ,  $\mathcal{A}$  cannot notice the change. Thus, we have

$$\text{Adv}_3 \leq \text{Adv}_4 + 2^{-\lambda} + 2^{-\delta}. \quad (22)$$

Note that the trivial cases are excluded in **TrivialU-Idp** and **TrivialCorr** of Fig. 3. Equation (22) is obtained according to the following non-trivial cases.

- Case 1:  $\mathcal{A}$  has issued **LeakPW**, but has not issued **LeakDK(U)** or **Corrupt(U)**. Without the PRF key  $k$ ,  $\mathcal{A}$ ’s advantage is upper bounded by  $2^{-\lambda}$ .
- Case 2:  $\mathcal{A}$  has issued **LeakDK**, but has not issued **LeakPW(U)**, **LeakFile(IS)**, or **Corrupt(U)**. In this case,  $pw$  is unknown to  $\mathcal{A}$ , and hence  $\mathcal{A}$ ’s advantage is upper bounded by  $2^{-\delta}$ .
- Case 3:  $\mathcal{A}$  has never issued any **LeakDK**, **LeakPW(U)**, **Corrupt(U)**, or **LeakFile(IS)**. Without  $pw$  and  $k$ ,  $\mathcal{A}$ ’s advantage is bounded by  $2^{-\lambda} + 2^{-\delta}$ .

**Game  $G_5$ .** In this game, we add an abort rule: if the signature  $\sigma_U$  is forged by  $\mathcal{A}$  in the target session, abort the game. By associating the  $\text{SIG.Sign}()$  oracle conceptually with the private key  $\text{sk}$ , a SIG-forgery can simulate the game for  $\mathcal{A}$  and undermine the EUF-CMA security of SIG with the probability that this game aborts. Therefore, we can get

$$\text{Adv}_4 \leq \text{Adv}_5 + \epsilon_{\text{SIG}}. \quad (23)$$

**Game  $G_6$ .** In this game, we add another abort rule: if  $\mathcal{A}$  ever queries  $H_3()$  with a value  $(z, \text{"MAC"})$  such that  $(pk_1, pk'_1, z)$  is a DH-triple, abort the game.

We claim that the probability of abort is upper bounded by  $\text{Adv}_{\mathcal{B}_1}^{\text{GCDH}}$ , where  $\mathcal{B}_1$  is an adversary against the GCDH problem  $(g^a, g^b)$ .  $\mathcal{B}_1$  simulates the game for  $\mathcal{A}$  as follows.

- $\mathcal{B}_1$  chooses all necessary long-term secrets.
- For the target session,  $\mathcal{B}_1$  sets  $pk_1 := g^a$  and  $pk'_1 := g^b$ , and  $\mathcal{B}_1$  chooses  $k_1 \xleftarrow{\$} \mathcal{K}$  for the target session.
- $\mathcal{B}_1$  simulates all random oracles as in Game  $G_1$  and observes all  $H_3(*, \text{"MAC"})$ -queries.
- $\mathcal{B}_1$  queries its own  $\mathcal{O}_{\text{DDH}}()$  with  $(pk_1, pk'_1, z)$  for every  $(z, \text{"MAC"})$  queried by  $\mathcal{A}$ . If  $\mathcal{O}_{\text{DDH}}$  returns 1 on any  $(pk_1, pk'_1, z)$ ,  $\mathcal{B}_1$  aborts the simulation and outputs this  $z$ .

If the abort occurs,  $\mathcal{B}_1$  simulates  $G_6$  for  $\mathcal{A}$ . Otherwise,  $\mathcal{B}_1$  simulates  $G_5$  for  $\mathcal{A}$ . Therefore, we have

$$\text{Adv}_5 \leq \text{Adv}_6 + \text{Adv}_{\mathcal{B}_1}^{\text{GCDH}}. \quad (24)$$

**Game  $G_7$ .** We change  $k_{1,\text{MAC}}$  to a random value. If Game  $G_6$  does not abort, then  $k_1$  is unknown to  $\mathcal{A}$ . Due to the random oracle  $H_3()$ ,  $H_3(k_1, *)$  is uniformly random in  $\mathcal{A}$ 's view and so is  $k_{1,\text{MAC}}$ . Thus, we have

$$\text{Adv}_6 = \text{Adv}_7. \quad (25)$$

**Game  $G_8$ .** In this game, if any forgery occurs on  $\text{mT}_1$ , abort the game. By associating the  $\text{MAC.Mac}()$  oracle conceptually with key  $k_{1,\text{MAC}}$ , a MAC-forgery can simulate the game for  $\mathcal{A}$  and break the EUF-CMA security of MAC with the probability that this game aborts. Hence we have

$$\text{Adv}_7 \leq \text{Adv}_8 + \epsilon_{\text{MAC}}. \quad (26)$$

**Game  $G_9$ .** In this game, another abort rule is added: if  $\mathcal{A}$  ever queries  $H_4()$  with a value  $H_4(z, (\text{ID}_U, \text{ID}_{AS}, \text{ID}_{IS}, pk_1, \text{ct}_U, pk'_1))$  such that  $(pk_1, pk'_1, z)$  is a DH-triple, and  $(\text{ID}_U, \text{ID}_{AS}, \text{ID}_{IS}, pk_1, \text{ct}_U, pk'_1)$  belongs to the target session, abort the game. Similar to Game  $G_6$ , there exists a GCDH adversary  $\mathcal{B}_2$  such that

$$\text{Adv}_8 \leq \text{Adv}_9 + \text{Adv}_{\mathcal{B}_2}^{\text{GCDH}}. \quad (27)$$

**Game**  $G_{10}$ . In this game, we replace  $\text{SK}_1$  with a uniformly random value in the key space. If Game  $G_9$  does not abort, then  $k_1$  is unknown to  $\mathcal{A}$ . It follows that the hash output is uniformly random in  $\mathcal{A}$ 's view and so is  $\text{SK}_1$ , due to the random oracle  $\text{H}_4()$ . Thus, we have

$$\text{Adv}_9 = \text{Adv}_{10}. \quad (28)$$

Now,  $\text{SK}_1$  becomes independent of  $b$ , all secret inputs, and randomness observable by  $\mathcal{A}$ , and hence we have

$$\text{Adv}_{10} = \Pr[G_{\mathcal{A},10} \Rightarrow 1] - \frac{1}{2} = \frac{1}{2} - \frac{1}{2} = 0. \quad (29)$$

Combining all (in)equalities from (18) to (29), we have proved Theorem 3.  $\square$

## E Proof of Theorem 4

*Proof.* This theorem is proved via a sequence of games.

**Game**  $G_0$ . This game is the original game as shown in the game  $\mathbf{G}_{\mathcal{A},\Sigma}^{\text{key-ind}}(1^\lambda)$ . Thus, we have

$$\text{Adv}_{\mathcal{A},\Sigma_{\text{AKE}}^{\text{B}}}^{\text{key}} + \frac{1}{2} = \Pr[\mathbf{G}_{\mathcal{A},\Sigma}^{\text{key-ind}}(1^\lambda) \Rightarrow 1] = \Pr[G_0 \Rightarrow 1] = \text{Adv}_0 + \frac{1}{2}. \quad (30)$$

**Game**  $G_1$ . In this game, we simulate all hash oracles  $\text{H}_\varsigma()$  ( $\varsigma = 2, 3, 4$ ) by maintaining hash lists  $L_{\text{H}}$ . According to the simulation,  $G_0$  and  $G_1$  are perfectly indistinguishable:

$$\text{Adv}_0 = \text{Adv}_1. \quad (31)$$

**Game**  $G_2$ . This game is the same as Game  $G_1$ , except that we add an abort rule: if collisions among outputs of hash queries and those among partial transcripts (e.g.,  $\text{ct}_3$  and  $\text{ct}'_3$ ) occur, abort the game. Thus, we can have

$$\text{Adv}_1 \leq \text{Adv}_2 + \frac{(q_{\text{Send}})^2}{2^{L_{\text{H}}}} + \frac{q_{\text{H}}^2}{2^{L_{\text{H}}}}. \quad (32)$$

**Game**  $G_3$ . In this game, we let the challenger first guess which is the target session  $(\pi_i^s, \pi_j^t)$ , and abort the game if the guess is wrong. This leads to

$$\text{Adv}_2 \leq (d\ell)^2 \cdot \text{Adv}_3. \quad (33)$$

**Game  $G_4$ .** In this game, we change the shared message  $ik$  to a random value for  $\pi_i^s$  and  $\pi_j^t$ . Note that if  $\mathcal{A}$  does not fail trivially (See  $\text{TrivAKE}()$  in Fig. 3),  $\mathcal{A}$  knows nothing about  $ik$  before the acceptance of  $\pi_i^s$  and  $\pi_j^t$ . Due to the key indistinguishability of  $\Sigma_{\text{AuthU}}^B$  and  $\Sigma_{\text{AuthAS}}^B$ , we have

$$\text{Adv}_3 \leq \text{Adv}_4 + \text{Adv}_{\mathcal{D}_1, \Sigma_{\text{AuthU}}^B}^{\text{key}} + \text{Adv}_{\mathcal{D}_2, \Sigma_{\text{AuthAS}}^B}^{\text{key}}. \quad (34)$$

**Game  $G_5$ .** In this game, we add another abort rule: if either  $\text{ct}_3$  or  $\text{ct}'_3$  is forged by the adversary in the target session, and either  $\pi_i^s$  or  $\pi_j^t$  accepts upon receiving them, abort the game. Due to the authenticity of AEAD, this change leads to

$$\text{Adv}_4 \leq \text{Adv}_5 + \epsilon_{\text{AEAD}}. \quad (35)$$

**Game  $G_6$ .** : This game is identical to the previous one, but we add another abort rule: if  $\mathcal{A}$  ever queries  $H_3()$  with a value  $(z, \text{"MAC"})$  such that  $(pk_3, pk'_3, z)$  is a DH-triple, abort the game.

We claim that the probability of abort is upper bounded by  $\text{Adv}_{\mathcal{B}_1}^{\text{GCDH}}$ , where  $\mathcal{B}_1$  is an adversary against the GCDH problem  $(g^a, g^b)$ .  $\mathcal{B}_1$  simulates the AKE game for  $\mathcal{A}$  as follows.

- $\mathcal{B}_1$  chooses all necessary long-term secrets and the shared secrets  $\{ik\}$ .
- For the target session,  $\mathcal{B}_1$  sets  $pk_3 := g^a$  and  $pk'_3 := g^b$ , and  $\mathcal{B}_1$  chooses  $\text{ms} \xleftarrow{\$} \mathcal{K}$  for the target session.
- $\mathcal{B}_1$  simulates all random oracles as in Game  $G_1$  and observes all  $H_3(*, \text{"MAC"})$ -queries.
- $\mathcal{B}_1$  queries its own  $\mathcal{O}_{\text{DDH}}()$  with  $(pk_3, pk'_3, z)$  for every  $(z, \text{"MAC"})$  queried by  $\mathcal{A}$ . If  $\mathcal{O}_{\text{DDH}}$  returns 1 on any  $(pk_3, pk'_3, z)$ ,  $\mathcal{B}_1$  aborts the simulation and output this  $z$ .

If the abort occurs,  $\mathcal{B}_1$  simulates  $G_6$  for  $\mathcal{A}$ . Otherwise,  $\mathcal{B}_1$  simulates  $G_5$  for  $\mathcal{A}$ . Therefore, we have

$$\text{Adv}_5 \leq \text{Adv}_6 + \text{Adv}_{\mathcal{B}_1}^{\text{GCDH}}. \quad (36)$$

**Game  $G_7$ .** We now change  $k_{\text{MAC}}$  to a random value. Note that if Game  $G_6$  does not abort, then  $\text{ms}$  is unknown to  $\mathcal{A}$ . In consequence,  $H_3(\text{ms}, *)$  is uniformly random in  $\mathcal{A}$ 's view and so is  $k_{\text{MAC}}$ , due to the random oracle  $H_3()$ . Thus, we have

$$\text{Adv}_6 = \text{Adv}_7. \quad (37)$$

**Game  $G_8$ .** We now add another abort rule: If any forgery occurs on  $\mathbf{mT}_3$  or  $\mathbf{mT}_4$ , abort the game. By associating the  $\text{MAC.Mac}()$  oracle conceptually with key  $k_{\text{MAC}}$ , a MAC-forgery can simulate the game for  $\mathcal{A}$  and break the EUF-CMA security of MAC with the probability that this game aborts. Thus, we have

$$\text{Adv}_7 \leq \text{Adv}_8 + \epsilon_{\text{MAC}}. \quad (38)$$

**Game  $G_9$ .** Another abort rule is added in this game: if  $\mathcal{A}$  ever queries  $H_4()$  with a value  $H_4(z, \text{ID}_U, \text{ID}_{AS}, \text{ct}_3, \text{ct}'_3)$  such that  $(pk_3, pk'_3, z)$  is a DH-triple, and  $\text{ID}_U, \text{ID}_{AS}, \text{ct}_3, \text{ct}'_3$  belongs to the target session, abort the game. Similar to Game  $G_6$ , there exists a GCDH adversary  $\mathcal{B}_2$  such that

$$\text{Adv}_8 \leq \text{Adv}_9 + \text{Adv}_{\mathcal{B}_2}^{\text{GCDH}}. \quad (39)$$

**Game  $G_{10}$ .** We now change  $\text{SK}_{AS}$  to a uniformly random value in the key space. Note that if Game  $G_9$  does not abort, then  $\text{ms}$  is unknown to  $\mathcal{A}$ . The hash output is uniformly random in  $\mathcal{A}$ 's view and so is  $\text{SK}_{AS}$ , due to the random oracle  $H_4()$ . Thus, we have

$$\text{Adv}_9 = \text{Adv}_{10}. \quad (40)$$

Moreover, as  $\text{SK}_{AS}$  becomes independent of  $b$ , all secret inputs and randomness observable by  $\mathcal{A}$ , we also have

$$\text{Adv}_{10} = \Pr[G_{\mathcal{A},10} \Rightarrow 1] - \frac{1}{2} = \frac{1}{2} - \frac{1}{2} = 0. \quad (41)$$

By collecting all (in)equalities from (30) to (41), we have proved Theorem 4.  $\square$

## F Proof of Theorem 5

*Proof.* We prove this theorem using a sequence of games.

**Game  $G_0$ .** This is the original game, so we have

$$\text{Adv}_{\mathcal{A}, \Sigma_{\text{AuthU}}^B, \Sigma_{\text{AKE}}^B}^{\text{PW}} = \Pr[\mathbf{G}_{\mathcal{A}, \Sigma}^{\text{PW-recov}}(1^\lambda) \Rightarrow 1] = \Pr[G_0 \Rightarrow 1]. \quad (42)$$

**Game  $G_1$ .** We add an abort rule here: when any collision of hash queries occurs, abort the game. As the numbers of each type of parties are upper bounded by  $d$  and the number of sessions per party is upper bounded by  $\ell$ , we have

$$\Pr[G_0 \Rightarrow 1] \leq \Pr[G_1 \Rightarrow 1] + \frac{(d\ell)^2}{2^{L_H}}. \quad (43)$$

**Game  $G_2$ .** We replace  $\text{PRF}()$  with a random function. This leads to

$$\Pr[G_1 \Rightarrow 1] \leq \Pr[G_2 \Rightarrow 1] + \epsilon_{\text{PRF}}. \quad (44)$$

**Game  $G_3$ .** Let  $\pi_U^s$  and  $\pi_S^t$  be two partnered oracles. We add an abort rule here: if there exists a  $\Sigma_{\text{AuthU}}^B$  session where  $\text{ct}_U$  is not honestly generated but either  $\pi_U^s$  and  $\pi_S^t$  enters **accept**, and the trivial cases are not triggered, abort the game. This event means  $\mathcal{A}$  can find a (new) ciphertext which can be decrypted to the same plaintext of an honestly generated ciphertext in that fresh session. This probability is upper bounded by the IND-CCA security of PKE. Since there are at most  $(d\ell)^2$  sessions, we have

$$\Pr[G_2 \Rightarrow 1] \leq \Pr[G_3 \Rightarrow 1] + (d\ell)^2 \cdot \epsilon_{\text{PKE}}. \quad (45)$$

**Game  $G_4$ .** In this game, we change every session key in the authentication phase to random values, and encrypt a randomly chosen value for each shared key encrypted. We claim that

$$\Pr[G_3 \Rightarrow 1] \leq \Pr[G_4 \Rightarrow 1] + (d\ell)^2 \text{Adv}_{\mathcal{D}_1, \Sigma_{\text{AuthU}}^B}^{\text{key}}. \quad (46)$$

This can be proved by a hybrid. Let  $G_4^i, i \in \{0, 1, \dots\}$  be the game where the first till the  $i$ -th established session key(s) have been set to random values in the authentication phase. If  $\mathcal{A}$  can distinguish  $G_4^i$  and  $G_4^{i+1}$ , then there exists  $\mathcal{D}_1$  against  $\Sigma_{\text{AuthU}}^B$ .

Note that in  $\mathbf{G}_{\mathcal{A}, \Sigma}^{\text{pw-recov}}(1^\lambda)$  defined in Fig. 2, the predicates in line 3-4 forbid the adversary to issue any **Corrupt()** or combinations of **LeakFile()**, **LeakPW()**, and **LeakDK()** to the user and IS parties. Therefore,  $\mathcal{D}_1$  does not fail trivially, and other queries can be simulated by  $\mathcal{D}_1$  with the queries provided by  $\mathcal{D}_1$ 's challenger.  $\mathcal{D}_1$  can issue **Test()** corresponding to the  $i+1$ -th session and forward the response as the result of  $\mathcal{A}$ 's **RevealKey()** to  $\mathcal{A}$ . If  $\mathcal{A}$  gives the correct password, which can be checked by using **Corrupt()**,  $\mathcal{D}_1$  returns 0. Otherwise  $\mathcal{D}_1$  returns 1.

If  $b = 1$ , **Test()** returns a random key, and  $\mathcal{D}_1$  simulates  $G_4^i$  for  $\mathcal{A}$ . If  $b = 0$ , **Test()** returns the real session key, and  $\mathcal{D}_1$  simulates  $G_4^{i+1}$  for  $\mathcal{A}$ . Therefore, if  $\mathcal{A}$  performs differently in  $G_4^i$  and  $G_4^{i+1}$  with non-negligible probability,  $\mathcal{D}_1$  can also efficiently distinguish the two keys, breaking the key indistinguishability of  $\Sigma_{\text{AuthU}}^B$ . As  $i \leq (d\ell)^2$ , we have proved (46) with a union bound.

**Game  $G_5$ .** In this game, we change the AEAD ciphertext in all the authentication phase protocol to the encryption of  $\{0\}^{\ell_\tau}$ . We claim that

$$\Pr[G_4 \Rightarrow 1] \leq \Pr[G_5 \Rightarrow 1] + (d\ell)^2 \epsilon_{\text{AEAD}}. \quad (47)$$

This can be proved by a hybrid as in Game  $G_4$ .

**Game  $G_6$ .** In this game, we change session keys in the AKE phase to a random values of the same length. We claim that

$$\Pr[G_5 \Rightarrow 1] \leq \Pr[G_6 \Rightarrow 1] + (d\ell)^2 \text{Adv}_{\mathcal{D}_2, \Sigma_{\text{AKE}}^{\text{B}}}^{\text{key}}. \quad (48)$$

This can be proved by a hybrid similar to Game  $G_4$ .

**Game  $G_7$ .** Till this game, the transcript, established fresh session keys and the fresh shared information have become independent of the passwords. As a password has minimal entropy of  $\delta$  and there are at most  $(d\ell)^2$  sessions, with a union bound, we have

$$\Pr[G_7 \Leftarrow 1] \leq (d\ell)^2 \cdot 2^{-\delta}. \quad (49)$$

By collecting the (in)equalities from (42) to (49), we can prove Theorem 5.  $\square$

## G Proof of Theorem 6

We consider two cases: the first oracle  $\pi_i^s$  that accepts maliciously satisfies  $\pi_i^s.\text{role} = \text{U}$  or  $\pi_i^s.\text{role} = \text{IS}$ . Theorem 6 is proved via two lemmas, each of which bounds the probability that  $\mathcal{A}$  succeeds in one case. The two probabilities are denoted by  $\text{Adv}_{\text{U}, \Sigma_{\text{AuthU}}^{\text{B}}}^{\text{auth}}$  and  $\text{Adv}_{\text{IS}, \Sigma_{\text{AuthU}}^{\text{B}}}^{\text{auth}}$ , and we have  $\text{Adv}_{\mathcal{A}, \Sigma_{\text{AuthU}}^{\text{B}}}^{\text{auth}} \leq \text{Adv}_{\text{U}, \Sigma_{\text{AuthU}}^{\text{B}}}^{\text{auth}} + \text{Adv}_{\text{IS}, \Sigma_{\text{AuthU}}^{\text{B}}}^{\text{auth}}$ .

**Lemma 1.** *For any adversary  $\mathcal{A}$  with running time  $t$ , the probability  $\text{Adv}_{\text{U}, \Sigma_{\text{AuthU}}^{\text{B}}}^{\text{auth}}$  that there is an oracle  $\pi_i^s$  with  $\pi_i^s.\text{role} = \text{U}$  that accepts maliciously is at most*

$$\text{Adv}_{\text{U}, \Sigma_{\text{AuthU}}^{\text{B}}}^{\text{auth}} \leq \frac{(q_{\text{Send}})^2 + q_{\text{H}}^2}{2^{L_{\text{H}}}} + d\ell(\epsilon_{\text{PKE}} + \frac{q_{\text{Send}}}{p} + \text{Adv}_{B_1}^{\text{GCDH}} + \epsilon_{\text{MAC}}). \quad (50)$$

*Proof.* The proof is proved via a sequence of games. Let “break <sub>$n$</sub> ” be the event that occurs when the first oracle  $\pi_i^s$  with  $\pi_i^s.\text{role} = \text{U}$  accepts maliciously in Game  $G_n$ .

**Game  $G_0$ .** This game is the original game. Thus, we have

$$\Pr[\text{break}_0] = \text{Adv}_{\text{U}, \Sigma_{\text{AuthU}}^{\text{B}}}^{\text{auth}}. \quad (51)$$

**Game  $G_1$ .** In this game, all hash oracles are simulated by maintaining hash lists. Therefore, we have

$$\Pr[\text{break}_0] = \Pr[\text{break}_1]. \quad (52)$$



**Game  $G_2$ .** In this game, we add an abort rule: if collisions among outputs of hash queries and those among partial transcripts (e.g.,  $pk_1$  and  $ct_U$ ), abort the game. The probabilities of collisions can be bounded by the birthday paradox:

$$\Pr[\text{break}_1] \leq \Pr[\text{break}_2] + ((q_{\text{Send}})^2 + q_H^2)/2^{L_H}. \quad (53)$$

**Game  $G_3$ .** We let the challenger guess which oracle with  $\pi_i^s.\text{role} = U$  will be the first oracle to accept maliciously. If the guess is wrong, the game is aborted. We can get

$$\Pr[\text{break}_2] = d\ell \cdot \Pr[\text{break}_3]. \quad (54)$$

**Game  $G_4$ .** This game is aborted if there exists a session where  $ct_U$  leaks information about the plaintext (e.g.,  $pk'_1$ ), and the trivial cases are not triggered. Due to the IND-CCA security of PKE, we can obtain

$$\Pr[\text{break}_3] \leq \Pr[\text{break}_4] + \epsilon_{\text{PKE}}. \quad (55)$$

**Game  $G_5$ .** This game is identical to Game  $G_4$  except that we add an abort rule: if  $\mathcal{A}$  succeeds in guessing the correct  $pk'_1$ , abort the game. Since  $pk'_1$  is an element in group  $\mathbb{G}$  of order  $p$ , we can have

$$\Pr[\text{break}_4] \leq \Pr[\text{break}_5] + q_{\text{Send}}/p. \quad (56)$$

**Game  $G_6$ .** In this game, we add another abort rule: if  $\mathcal{A}$  ever queries  $H_3()$  with a value  $(k_1, \text{"MAC"})$  such that  $(pk_1, pk'_1, k_1)$  is a DH-triple, abort the game. As shown in Game  $G_5$  in the proof of Theorem 4, we have

$$\Pr[\text{break}_5] \leq \Pr[\text{break}_6] + \text{Adv}_{\mathcal{B}_1}^{\text{GCDH}}. \quad (57)$$

**Game  $G_7$ .** We change  $k_{1,\text{MAC}}$  to a random value. If Game  $G_6$  does not abort, then  $k_1$  is unknown to  $\mathcal{A}$ . Due to the random oracle  $H_3()$ ,  $H_3(k_1, *)$  is uniformly random in  $\mathcal{A}$ 's view and so is  $k_{1,\text{MAC}}$ . Thus, we have

$$\Pr[\text{break}_6] = \Pr[\text{break}_7]. \quad (58)$$

**Game  $G_8$ .** This game is identical to Game  $G_7$  except that it is aborted if any forgery occurs on  $mT_1$ . Due to the EUF-CMA security of MAC, we have

$$\Pr[\text{break}_7] \leq \Pr[\text{break}_8] + \epsilon_{\text{MAC}}. \quad (59)$$

In this game, all messages are honest, and the previous games exclude all forgeries and trivial cases, so that  $\pi_i^s$  will not accept maliciously. Hence we have

$$\Pr[\text{break}_8] = 0. \quad (60)$$

Combining the (in)equalities from (51) to (60), we have proved Lemma 1.  $\square$

**Lemma 2.** *For any adversary  $\mathcal{A}$  with running time  $t$ , the probability  $\text{Adv}_{\text{IS}, \Sigma_{\text{AuthU}}^{\text{B}}}^{\text{auth}}$  that there is an oracle  $\pi_i^s$  with  $\pi_i^s.\text{role} = \text{IS}$  that accepts maliciously is at most*

$$\text{Adv}_{\text{IS}, \Sigma_{\text{AuthU}}^{\text{B}}}^{\text{auth}} \leq \frac{(q_{\text{Send}})^2 + q_{\text{H}}^2}{2^{L_{\text{H}}}} + \epsilon_{\text{PRF}} + d\ell(\epsilon_{\text{SIG}} + \epsilon_{\text{PKE}}). \quad (61)$$

*Proof.* The proof proceeds in a sequence of games. Let “break <sub>$n$</sub> ” be the event that occurs when the first oracle  $\pi_i^s$  with  $\pi_i^s.\text{role} = \text{IS}$  accepts maliciously in Game  $G_n$ .

**Game  $G_0$ .** This game is the original game. Thus, we have

$$\Pr[\text{break}_0] = \text{Adv}_{\text{IS}, \Sigma_{\text{AuthU}}^{\text{B}}}^{\text{auth}}. \quad (62)$$

**Game  $G_1$ .** In this game, all hash oracles and PRF queries are simulated by maintaining lists. Therefore, we can get

$$\Pr[\text{break}_0] \leq \Pr[\text{break}_1] + \epsilon_{\text{PRF}}. \quad (63)$$

**Game  $G_2$ .** In this game, we add an abort rule: if collisions among outputs of hash (or PRF) queries and those among partial transcripts (e.g.,  $pk_1$  and  $ct_U$ ) occur, abort the game. Therefore, we can get

$$\Pr[\text{break}_1] \leq \Pr[\text{break}_2] + ((q_{\text{Send}})^2 + q_{\text{H}}^2)/2^{L_{\text{H}}}. \quad (64)$$

**Game  $G_3$ .** We let the challenger guess which oracle with  $\pi_i^s.\text{role} = \text{IS}$  will be the first oracle to accept maliciously. If the guess is wrong, the game is aborted. We can get

$$\Pr[\text{break}_2] = d\ell \cdot \Pr[\text{break}_3]. \quad (65)$$

**Game  $G_4$ .** This game is identical to Game  $G_3$ . The trivial cases are excluded by TrivialU-Idp and TrivialPw defined in Fig. 3. We now consider the remaining non-trivial cases.

- Case 1:  $\mathcal{A}$  has issued  $\text{LeakPW}(U)$  and  $\text{LeakFile}(\text{IS})$ , but has not issued  $\text{LeakDK}(U)$  or  $\text{Corrupt}(U)$ . Without the PRF key  $k$ ,  $sk_U$  is unknown to  $\mathcal{A}$  due to the random oracle. Therefore,  $\mathcal{A}$ ’s advantage can be bounded by  $\epsilon_{\text{SIG}}$  according to the EUF-CMA security of SIG.
- Case 2:  $\mathcal{A}$  has issued  $\text{LeakDK}(U)$ , but has not issued  $\text{LeakPW}(U)$ ,  $\text{LeakFile}(\text{IS})$ , or  $\text{Corrupt}(U)$ . Without the password  $pw$ ,  $sk_U$  is unknown to  $\mathcal{A}$  due to the random oracle. Thus,  $\mathcal{A}$ ’s advantage can be bounded by  $\epsilon_{\text{SIG}}$ .

- Case 3:  $\mathcal{A}$  has never issued any  $\text{LeakDK}(\mathbf{U})$ ,  $\text{LeakPW}(\mathbf{U})$ ,  $\text{Corrupt}(\mathbf{U})$ , or  $\text{LeakFile}(\mathbf{IS})$ .  $\mathcal{A}$ 's advantage can be bounded by  $\epsilon_{\text{SIG}} + \epsilon_{\text{PKE}}$ , where  $\epsilon_{\text{SIG}}$  is obtained as above, and  $\epsilon_{\text{PKE}}$  is obtained by the fact that  $\mathcal{A}$  cannot generate  $\text{ct}_{\mathbf{U}}$  s.t.  $\pi_i^s$  would accept.

Accordingly, we can have

$$\Pr[\text{break}_3] \leq \Pr[\text{break}_4] + \epsilon_{\text{SIG}} + \epsilon_{\text{PKE}}. \quad (66)$$

In this game, all messages are honest, and the previous games exclude all forgeries and trivial cases, so that  $\pi_i^s$  will not accept maliciously. Hence we have

$$\Pr[\text{break}_4] = 0. \quad (67)$$

Combining the (in)equalities from (62) to (67), we have proved Lemma 2.  $\square$

## H Proof of Theorem 7

We consider two cases: the first oracle  $\pi_i^s$  that accepts maliciously satisfies  $\pi_i^s.\text{role} = \mathbf{U}$  or  $\pi_i^s.\text{role} = \mathbf{AS}$ . Theorem 7 is proved via two lemmas, each of which bounds the probability that  $\mathcal{A}$  succeeds in one case. The two probabilities are denoted by  $\text{Adv}_{\mathbf{U}, \Sigma_{\text{AKE}}^{\text{B}}}^{\text{auth}}$  and  $\text{Adv}_{\mathbf{IS}, \Sigma_{\text{AKE}}^{\text{B}}}^{\text{auth}}$ , and we have  $\text{Adv}_{\mathcal{A}, \Sigma_{\text{AKE}}^{\text{B}}}^{\text{auth}} \leq \text{Adv}_{\mathbf{U}, \Sigma_{\text{AKE}}^{\text{B}}}^{\text{auth}} + \text{Adv}_{\mathbf{IS}, \Sigma_{\text{AKE}}^{\text{B}}}^{\text{auth}}$ .

**Lemma 3.** *For any adversary  $\mathcal{A}$  with running time  $t$ , the probability  $\text{Adv}_{\mathbf{U}, \Sigma_{\text{AKE}}^{\text{B}}}^{\text{auth}}$  that there is an oracle  $\pi_i^s$  with  $\pi_i^s.\text{role} = \mathbf{U}$  that accepts maliciously is at most*

$$\begin{aligned} \text{Adv}_{\mathbf{U}, \Sigma_{\text{AKE}}^{\text{B}}}^{\text{auth}} \leq & \frac{(q_{\text{Send}})^2 + q_{\text{H}}^2}{2L_{\text{H}}} + (d\ell)(\text{Adv}_{\mathcal{D}_1, \Sigma_{\text{AuthU}}^{\text{B}}}^{\text{key}} + \text{Adv}_{\mathcal{D}_2, \Sigma_{\text{AuthAS}}^{\text{B}}}^{\text{key}} \\ & + \epsilon_{\text{AEAD}} + \text{Adv}_{\mathcal{B}_1}^{\text{GCDH}} + \epsilon_{\text{MAC}}). \end{aligned} \quad (68)$$

*Proof.* The proof proceeds in a sequence of games. Let “ $\text{break}_n$ ” be the event that occurs when the first oracle  $\pi_i^s$  with  $\pi_i^s.\text{role} = \mathbf{U}$  accepts maliciously in Game  $G_n$ .

**Game  $G_0$ .** This game is the original game. Thus, we have

$$\Pr[\text{break}_0] = \text{Adv}_{\mathbf{U}, \Sigma_{\text{AKE}}^{\text{B}}}^{\text{auth}}. \quad (69)$$

**Game  $G_1$ .** In this game, we simulate all hash oracles  $\text{H}_{\varsigma}()$  ( $\varsigma = 2, 3, 4$ ) by maintaining hash lists. According to the simulation,  $G_0$  and  $G_1$  are perfectly indistinguishable:

$$\Pr[\text{break}_0] = \Pr[\text{break}_1]. \quad (70)$$

**Game  $G_2$ .** This game is the same as Game  $G_1$ , except that we add an abort rule: if collisions among outputs of hash queries and those among partial transcripts (e.g.,  $\text{ct}_3$  and  $\text{ct}'_3$ ) occur, abort the game. Therefore, we have

$$\Pr[\text{break}_1] \leq \Pr[\text{break}_2] + \frac{(q_{\text{Send}})^2 + q_{\text{H}}^2}{2^{L_{\text{H}}}}. \quad (71)$$

**Game  $G_3$ .** We let the challenger guess which oracle with  $\pi_i^s.\text{role} = \text{U}$  will be the first oracle to accept maliciously. If the guess is wrong, the game is aborted. We can get

$$\Pr[\text{break}_2] = d\ell \cdot \Pr[\text{break}_3]. \quad (72)$$

**Game  $G_4$ .** In this game, we change the shared message  $\text{ik}$  to a random value for  $\pi_i^s$  and  $\pi_j^t$ . Note that if  $\mathcal{A}$  does not fail trivially (See  $\text{TrivAKE}()$  in Fig. 3),  $\mathcal{A}$  knows nothing about  $\text{ik}$  before the acceptance of  $\pi_i^s$  and  $\pi_j^t$ . Due to the key indistinguishability of  $\Sigma_{\text{AuthU}}^{\text{B}}$  and  $\Sigma_{\text{AuthAS}}^{\text{B}}$ , we have

$$\Pr[\text{break}_3] \leq \Pr[\text{break}_4] + \text{Adv}_{\mathcal{D}_1, \Sigma_{\text{AuthU}}^{\text{B}}}^{\text{key}} + \text{Adv}_{\mathcal{D}_2, \Sigma_{\text{AuthAS}}^{\text{B}}}^{\text{key}}. \quad (73)$$

**Game  $G_5$ .** In this game, we add another abort rule: if there exists a session where  $\text{ct}'_3$  is forged by the adversary, abort the game. Due to the authenticity of AEAD, we have

$$\Pr[\text{break}_4] \leq \Pr[\text{break}_5] + \epsilon_{\text{AEAD}}. \quad (74)$$

**Game  $G_6$ .** In this game, we add another abort rule: if  $\mathcal{A}$  ever queries  $\text{H}_3()$  with a value  $(z, \text{"MAC"})$  such that  $(pk_3, pk'_3, z)$  is a DH-triple, abort the game. As shown in Game  $G_5$  in the proof of Theorem 4, we can obtain

$$\Pr[\text{break}_5] \leq \Pr[\text{break}_6] + \text{Adv}_{\mathcal{B}_1}^{\text{GCDH}}. \quad (75)$$

**Game  $G_7$ .** We change  $k_{\text{MAC}}$  to a random value. Note that if Game  $G_6$  does not abort, then  $\text{ms}$  is unknown to  $\mathcal{A}$ . It follows that  $\text{H}_3(\text{ms}, *)$  is uniformly random in  $\mathcal{A}$ 's view and so is  $k_{\text{MAC}}$ , due to the random oracle  $\text{H}_3()$ . Therefore, we have

$$\Pr[\text{break}_6] = \Pr[\text{break}_7]. \quad (76)$$

**Game  $G_8$ .** We now add another abort rule: If any forgery occurs on  $\mathbf{mT}_3$ , abort the game. Due to the EUF-CMA security of MAC, we have

$$\Pr[\text{break}_7] = \Pr[\text{break}_8] + \epsilon_{\text{MAC}}. \quad (77)$$

In this game, all messages are honest, and the previous games exclude all forgeries and trivial cases, so that  $\pi_i^s$  will not accept maliciously. Hence we have

$$\Pr[\text{break}_8] = 0. \quad (78)$$

Combining the (in)equalities from (69) to (78), we have proved Lemma 3.  $\square$

In addition, we can obtain  $\text{Adv}_{\text{IS}, \Sigma_{\text{AKE}}^{\text{B}}}^{\text{auth}} \leq \frac{(q_{\text{Send}})^2 + q_{\text{H}}^2}{2^{L_{\text{H}}}} + (dl)(\text{Adv}_{\mathcal{D}_1, \Sigma_{\text{AuthU}}^{\text{B}}}^{\text{key}} + \text{Adv}_{\mathcal{D}_2, \Sigma_{\text{AuthAS}}^{\text{B}}}^{\text{key}} + \epsilon_{\text{AEAD}} + \text{Adv}_{\mathcal{B}_1}^{\text{GCDH}} + \epsilon_{\text{MAC}})$ . The corresponding lemma and proof are identical to that of Lemma 3, and would be omitted.

## I Theoretical Computation Overhead

We estimate the computation overheads in terms of basic cryptographic operations. *Exp* is one modular exponentiation, *SigKg* one execution of SIG.Gen, *SigGen/SigVrf* a generation/verification operation of a signature, *PEnc/PDec* an encryption/decryption operation of PKE, *AEnc/ADec* an encryption/decryption operation of AEAD, and *Hash* one execution of PRF, MAC.Mac/MAC.Vry, or hash functions. We focus on evaluating computation overheads of the authentication and AKE phases, and omit those of the registration phase, as the registration is only performed once (for a user or an application server) and would not affect users' experience.

In  $\Sigma_{\text{AuthU}}^{\text{B}}$  of the basic SaPBC protocol  $\Sigma^{\text{B}}$ , a user U generates her/his signature key pair and an ephemeral key pair, and performs a signature operation, a public-key encryption, a Diffie-Hellman key agreement, one operation of MAC.Vry, two hash operations, and an AEAD decryption. The total computation costs of U are  $2 \cdot \text{Mul} + 4 \cdot \text{Hash} + \text{SigKg} + \text{SigGen} + \text{PEnc} + \text{ADec}$ . For the identity server IS, it produces an ephemeral key pair and executes a public-key decryption, a signature verification, a Diffie-Hellman key agreement, one operation of MAC.Mac, two hash operations, and an AEAD encryption. The whole computational overheads of IS are  $2 \cdot \text{Mul} + 3 \cdot \text{Hash} + \text{PDec} + \text{SigVrf} + \text{AEnc}$ . Compared with  $\Sigma_{\text{AuthU}}^{\text{B}}$ ,  $\Sigma_{\text{AuthU}}^{\text{E}}$  requires U to perform extra  $2 \cdot \text{Hash}$  operations.

In  $\Sigma_{\text{AuthAS}}^{\text{B}}$  (or  $\Sigma_{\text{AuthAS}}^{\text{E}}$ ), the application server AS performs the same operations as U does in  $\Sigma_{\text{AuthU}}^{\text{B}}$  except that the signature key pair is not needed to be re-generated. The computation overheads on the AS side are  $2 \cdot \text{Mul} + 3 \cdot \text{Hash} + \text{SigGen} + \text{PEnc} + \text{ADec}$ . For IS, the operations are almost the same as those in  $\Sigma_{\text{AuthU}}^{\text{B}}$ , where the sole difference is that the secret key *ik* is retrieved from the state **State** rather than generated via **AEAD.Gen**. Thus, its computation costs are  $2 \cdot \text{Mul} + 3 \cdot \text{Hash} + \text{PDec} + \text{SigVrf} + \text{AEnc}$ .

In  $\Sigma_{\text{AKE}}^{\text{B}}$ , U or AS executes an ephemeral key pair generation, an AEAD encryption, an AEAD decryption, a Diffie-Hellman key agreement, two hash operation, and one operation of MAC.Mac and MAC.Vry. The computational costs of U or AS are  $2 \cdot \text{Mul} + \text{AEnc} + \text{ADec} + 4\text{Hash}$ . By comparison,  $\Sigma_{\text{AKE}}^{\text{E}}$  requires each party to additionally perform two hash operations and one modular exponentiation.